

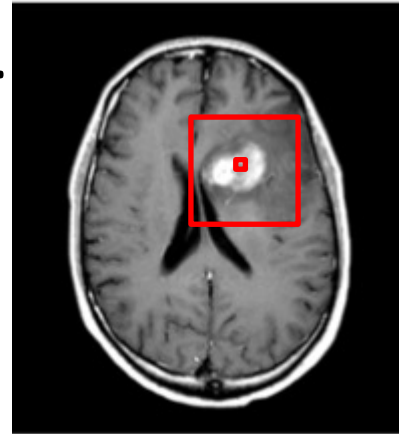
CPSC 340: Machine Learning and Data Mining

Convolutional Neural Networks

Last Time: Convolutions

→ make a filter and provide information about spaces

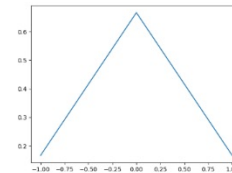
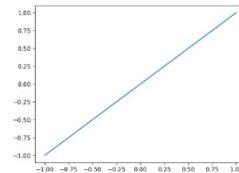
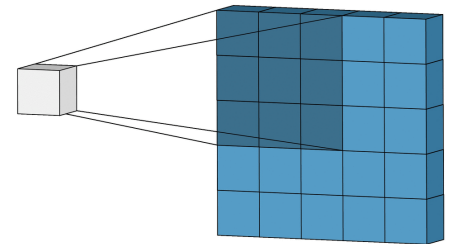
- We considered building a classifier to **classify pixels** in an image:
 - To represent “context” of a pixel we discussed using **convolutions**.



- Convolutions are **weighted combinations of the nearby pixels**.

$$z[i_1, i_2] = \sum_{j_1=-m}^m \sum_{j_2=-m}^m w[j_1, j_2] x[i_1 + j_1, i_2 + j_2]$$

- Can approximate image “**derivatives**” and “**integrals**”.

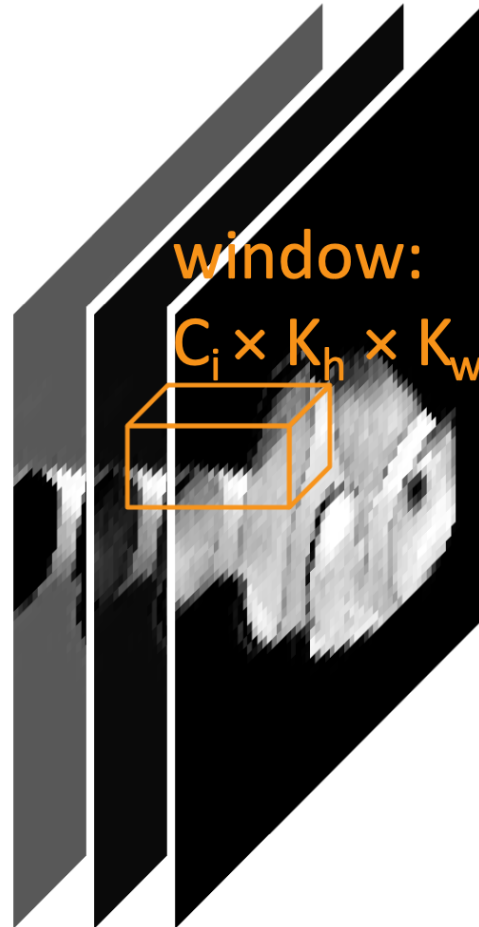


- At multiple **scales** and in multiple **directions**.

Next Topic: Multi-channel Inputs and Stride

Multi-channel Inputs

↪ Concept of input and output ∞



filter:
 $C_i \times K_h \times K_w$

$*$  $=$

Tensor of Filters

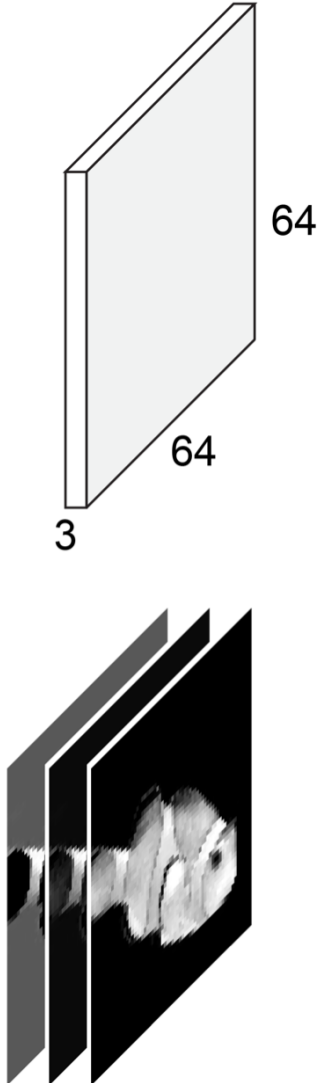
For each location,

you have C_i numbers, coming from



Filters.

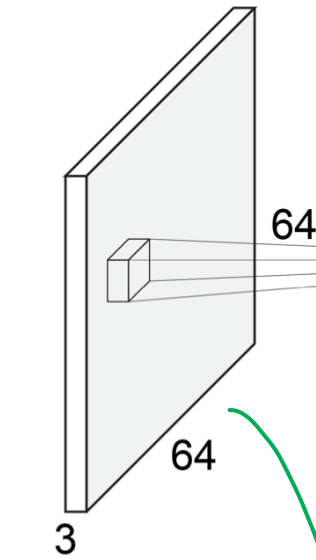
Tensor Views



- Tensor: high-dimension array
- feature maps
 - 3-D tensor: $C \times H \times W$
 - C: channels
 - H: height
 - W: width

Tensor Views

Channels $\rightarrow$ Features
for each
Pixel



1. First derivati
2. blurring
3. ...

For one Pixel
we have a vector
of 3 that we convert
them to a vector of 16

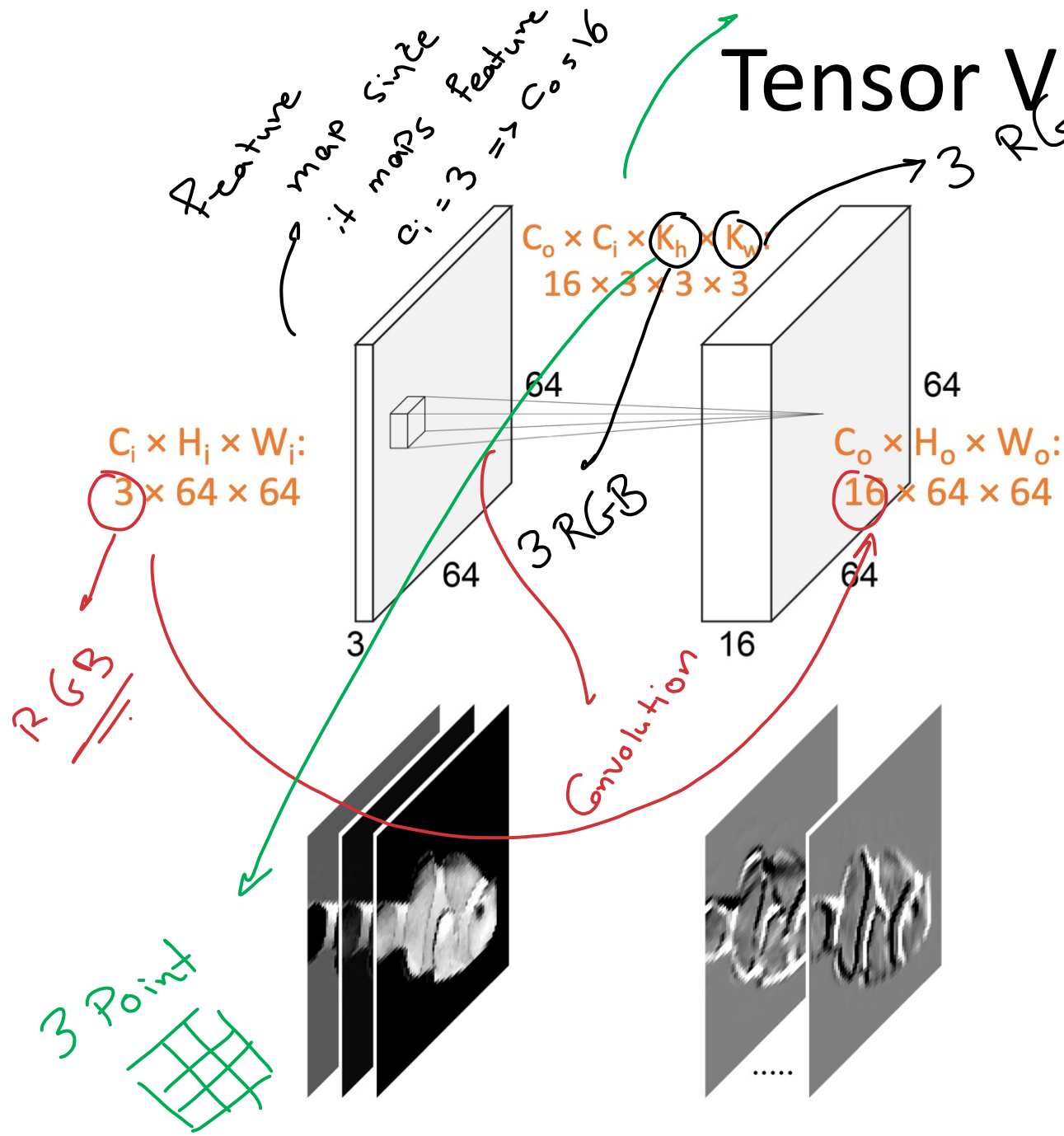
- Tensor: high-dimension array
- feature maps
 - 3-D tensor: $C \times H \times W$
 - C : channels
 - H : height
 - W : width
- filters
 - 4-D tensor: $C_o \times C_i \times K_h \times K_w$
 - C_o : output channels
 - C_i : input channels
 - K_h, K_w : filter height, width

Here

kernels

kernels

Tensor Views



Your Filter has one more dimension than your input and output; so, when input is 3 dimension $\rightarrow$ filter is 4

- Tensor: high-dimension array

- feature maps

- 3-D tensor: $C \times H \times W$
- C: channels
- H: height
- W: width

- filters

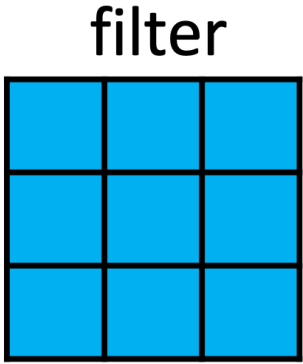
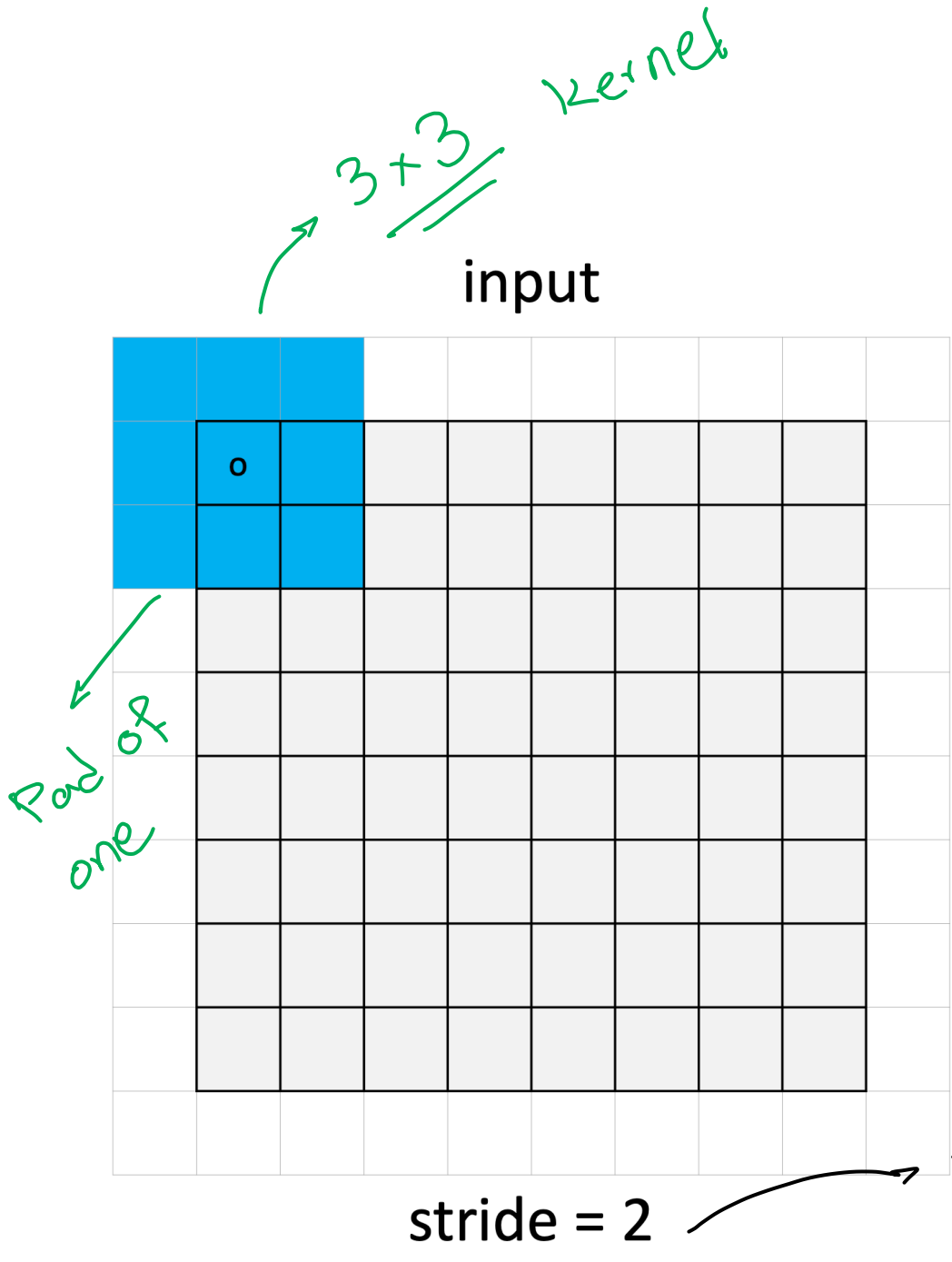
- 4-D tensor: $C_o \times C_i \times K_h \times K_w$
- C_o : output channels
- C_i : input channels
- K_h, K_w : filter height, width

Input (3 color RGB) input width

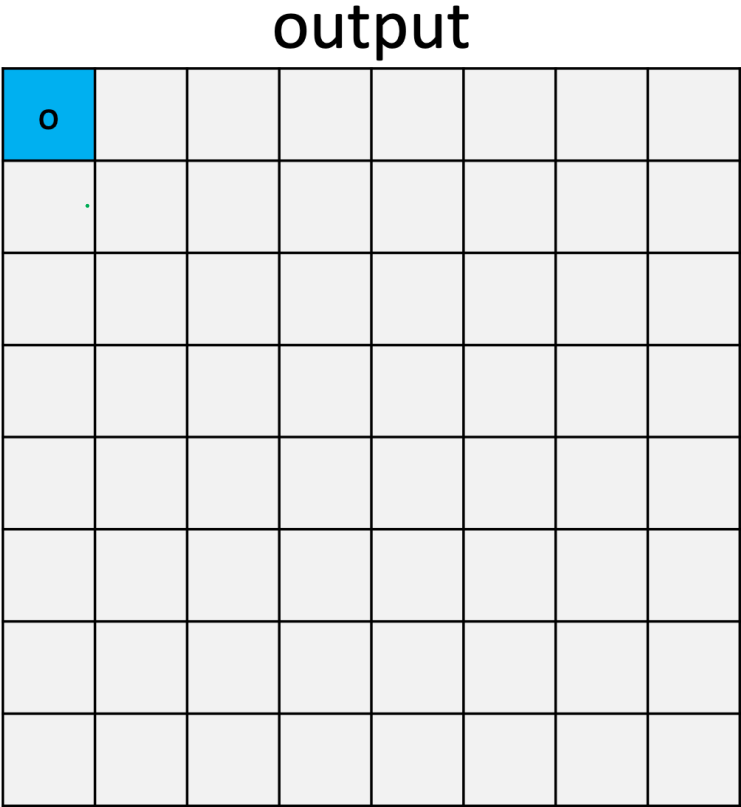
height

Stride

↓
Output

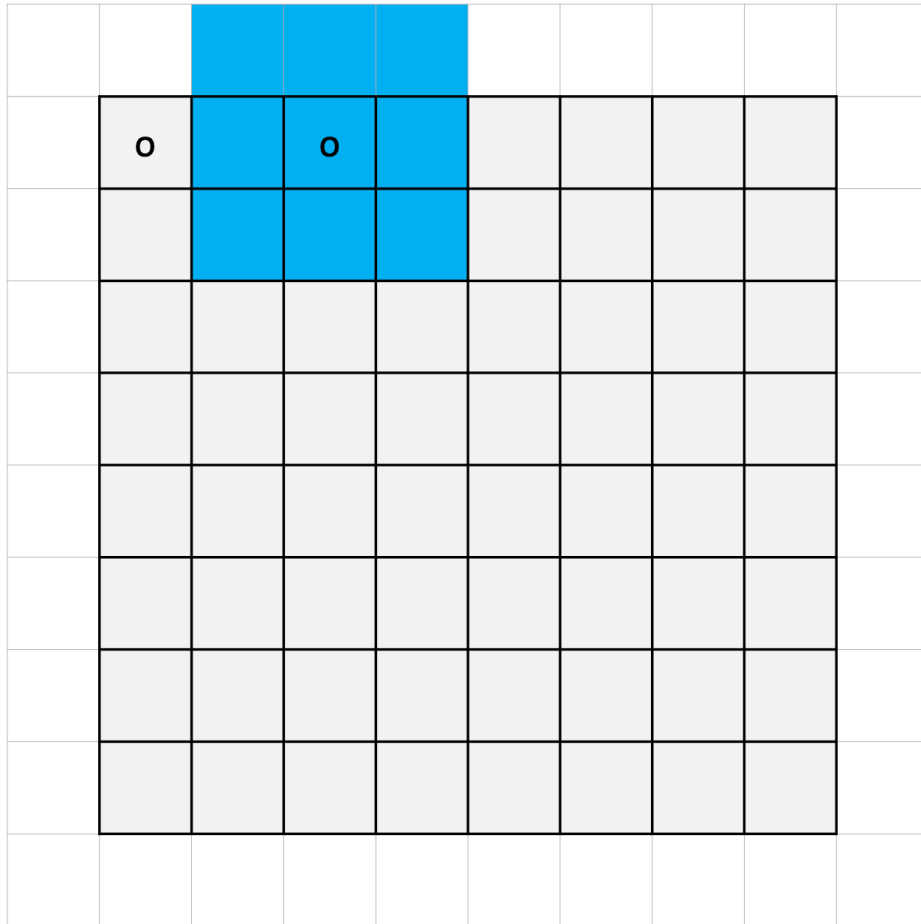


Jump by 2
Skip every other
Pixel

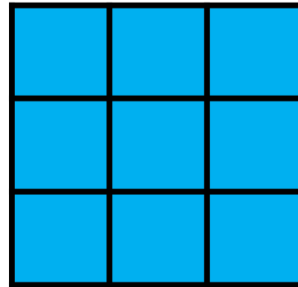


Stride

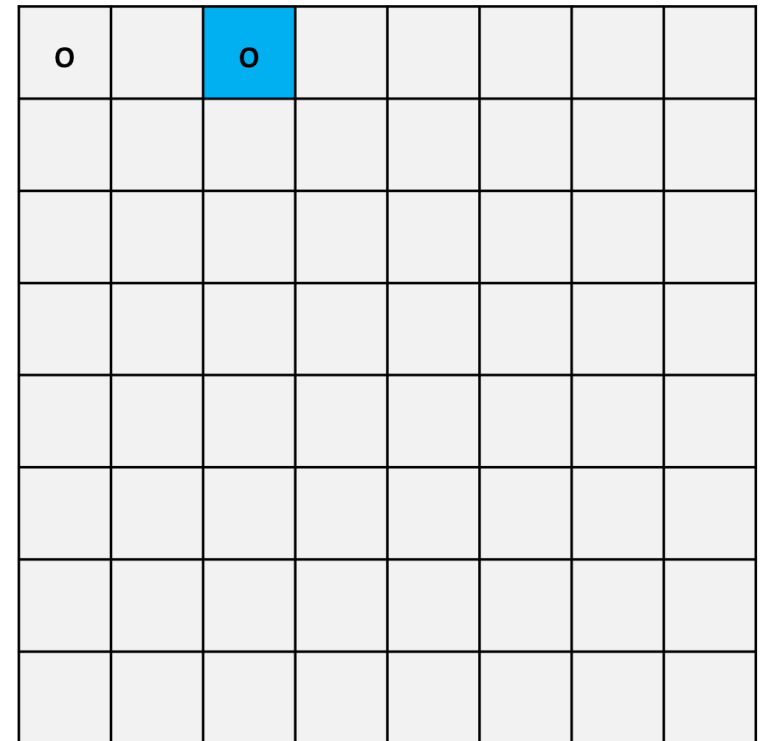
input



filter

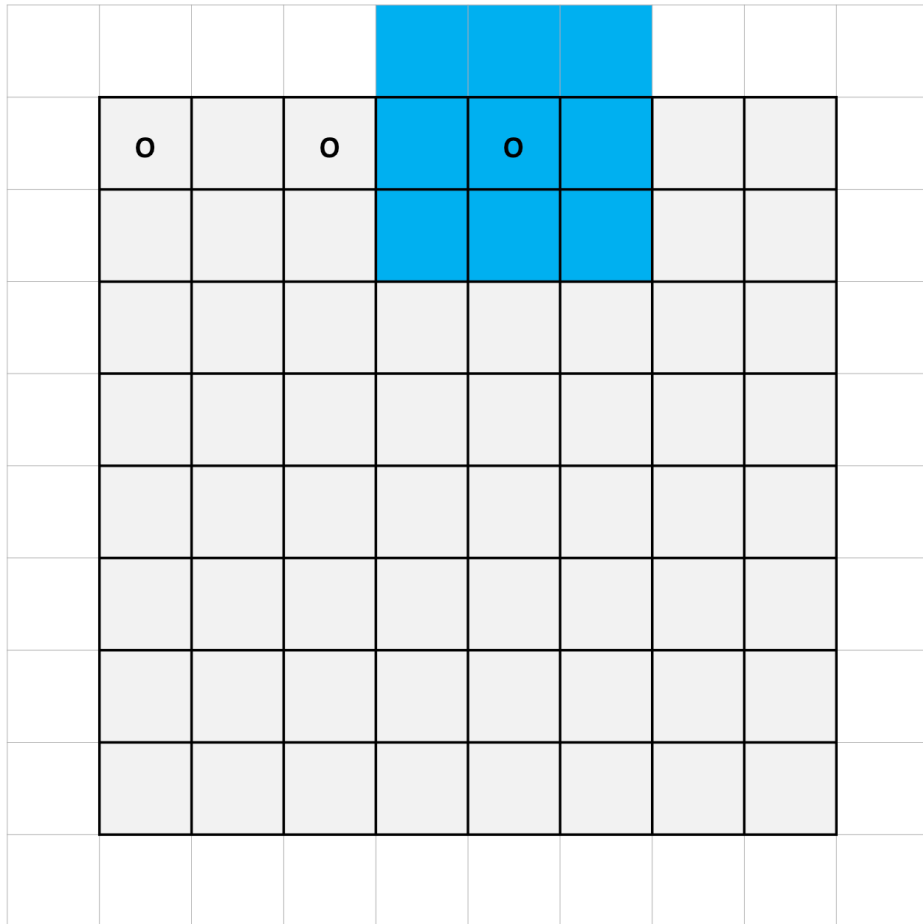


output



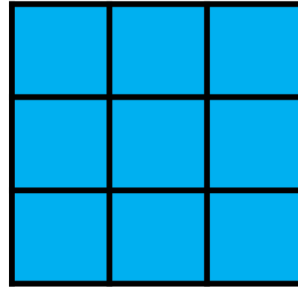
Stride

input

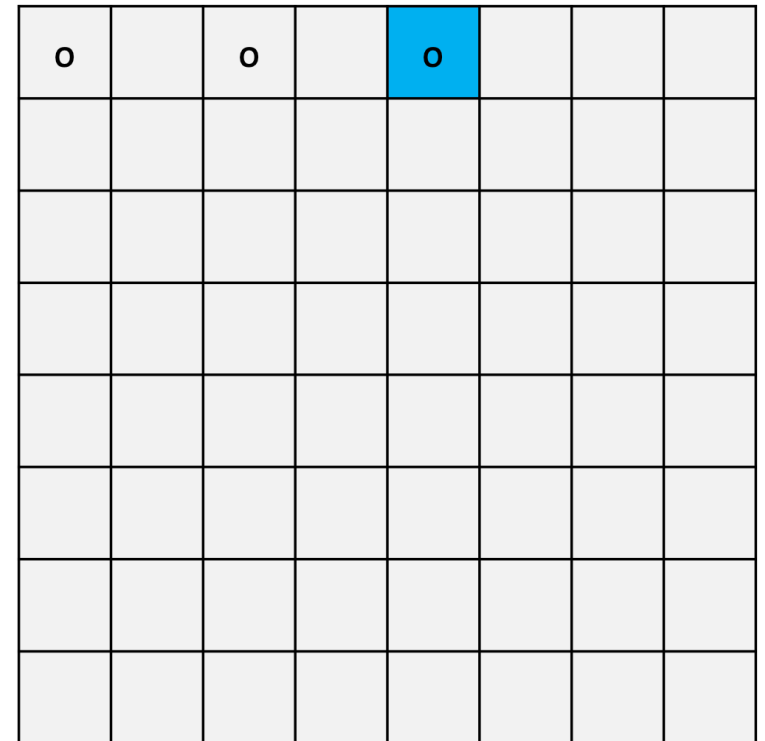


stride = 2

filter



output



Stride

input

[illegible]

stride = 2

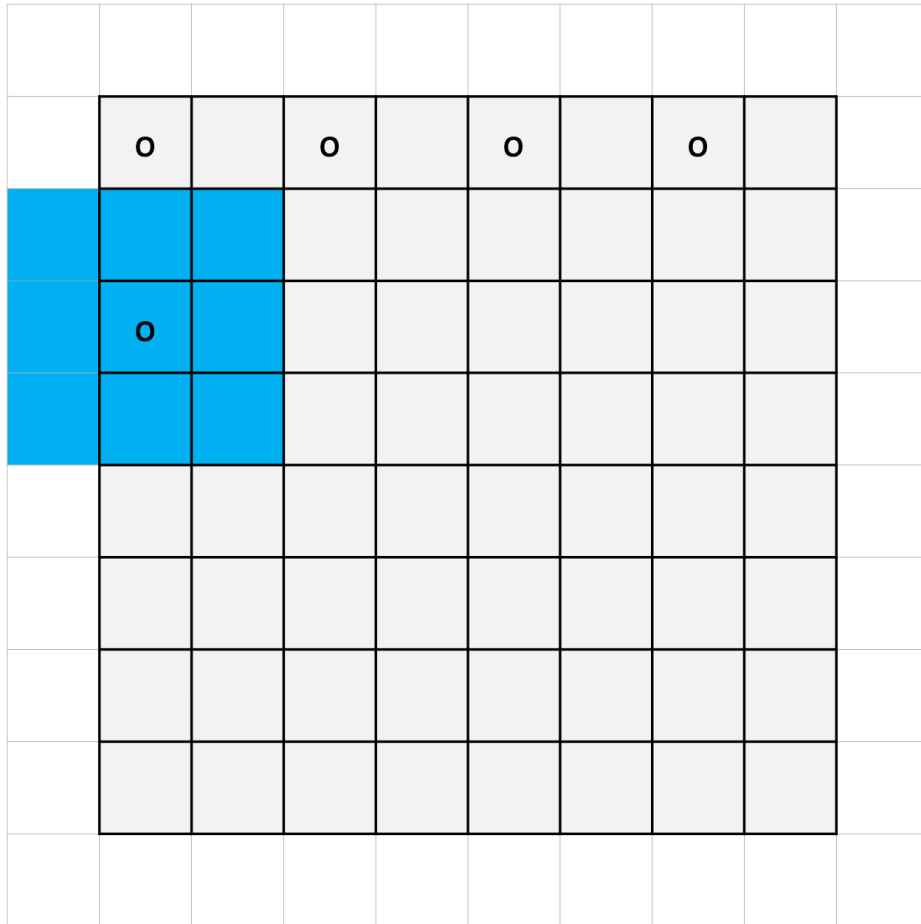
filter

output

[illegible]

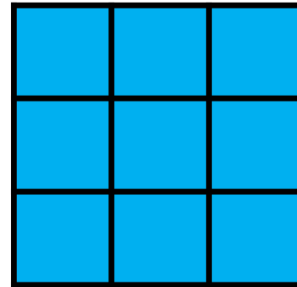
Stride

input

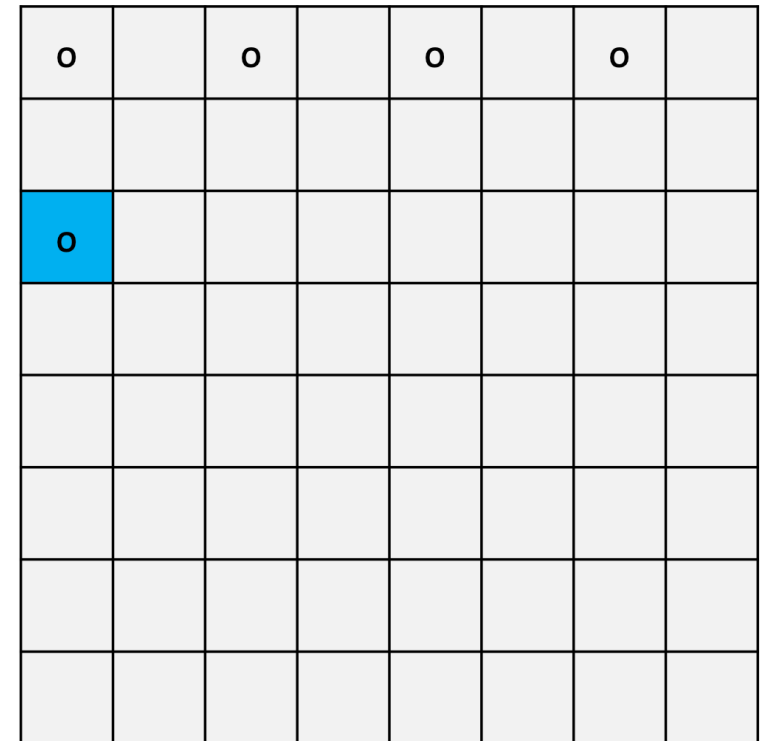


stride = 2

filter



output



Stride

input

	o		o		o		o		
	o		o		o		o		
	o		o		o		o		
	o		o		o		o		
	o		o		o		o		

filter

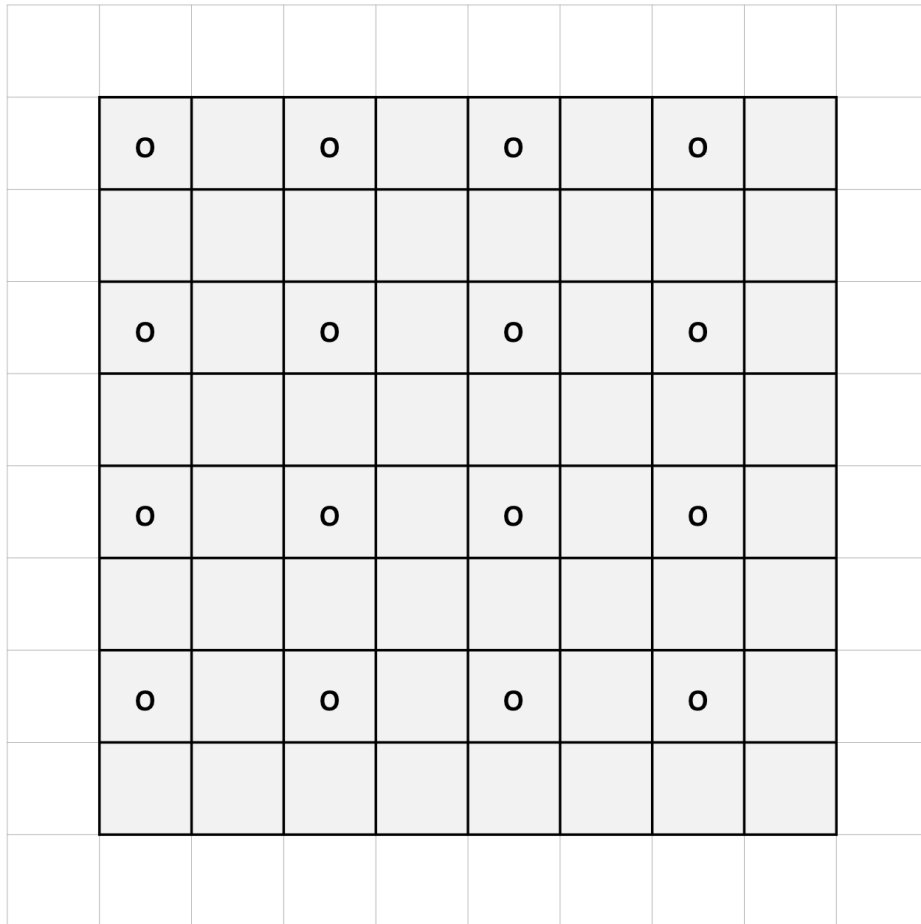
output

o		o		o		o	
o		o		o		o	
o		o		o		o	
o		o		o		o	

Stride

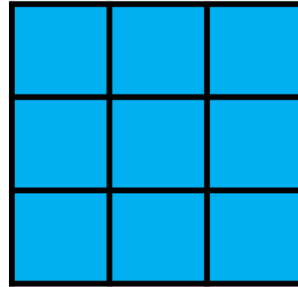
Dimension
is reduced by 4

input: $H \times W = 8 \times 8$

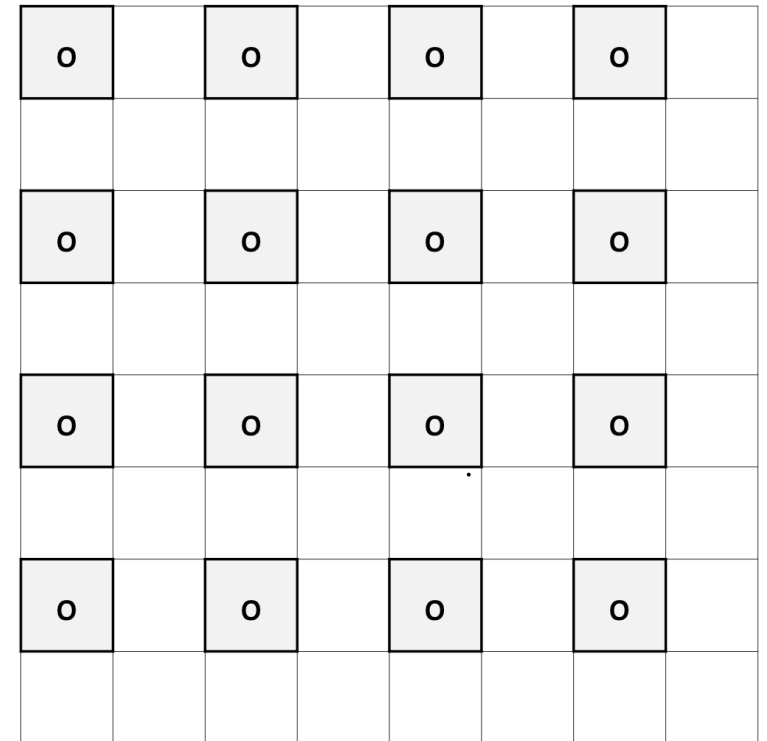


stride = 2

filter

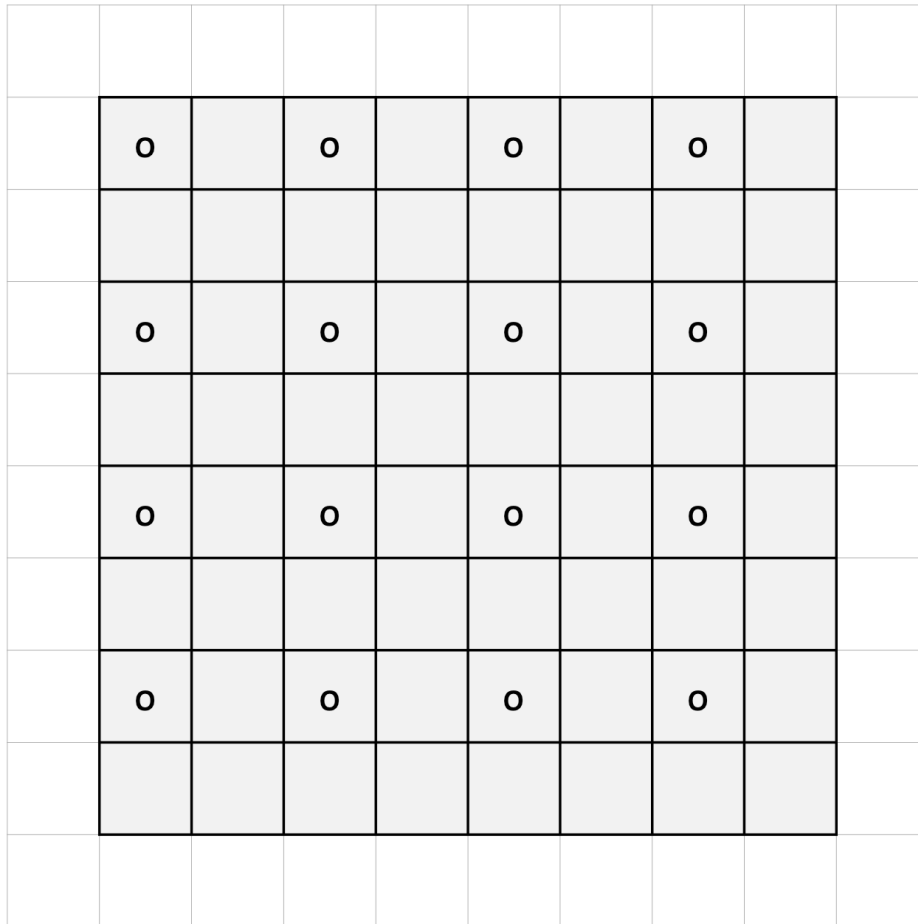


output: $H \times W = 4 \times 4$



Stride

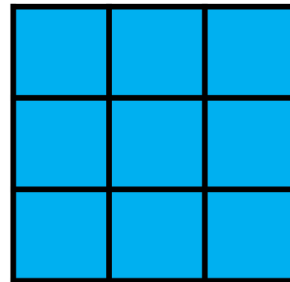
input: $H \times W = 8 \times 8$



stride = 2

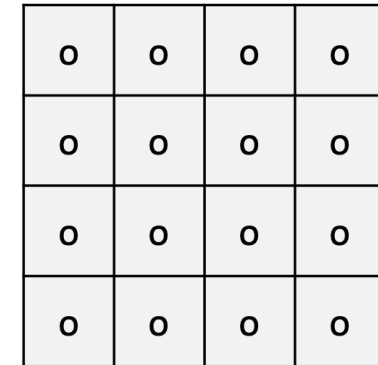
output: $H \times W = 4 \times 4$

filter



size of output

A green handwritten arrow points from this text to the output grid.



$$H_{\text{out}} = \lfloor (H_{\text{in}} + 2\text{pad}_h - K_h) / \text{str} \rfloor + 1$$

A handwritten arrow points from the pad_h term in the equation to the text "pad" below it.

pad

Major Convolution Arguments

- filter (kernel) size: $K_h \times K_w$ (e.g., 3×3)

- output channels: C_o
 - Input channels: C_i is determined by input

$3 + w + h$

- Padding (e.g., 1)
- Stride (e.g., 1 or 2)

↑ Multi-channel
RGB, for example

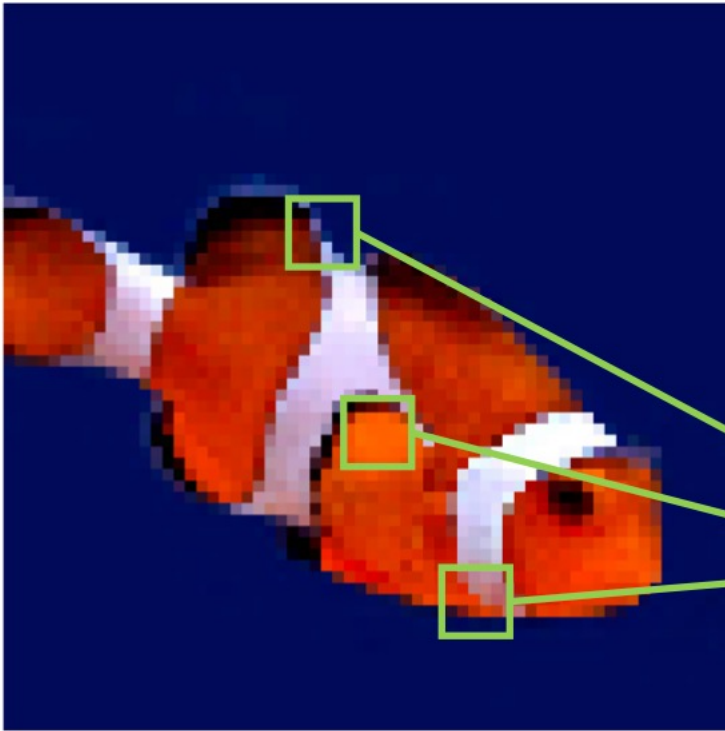
Next Topic: Convolution Properties



Translation-invariant

- Process each window in the same way

filters are translation-invariant



apply the same weights regardless of the window location

Translation-invariant

- Process each window in the same way

$$z[i_1, i_2] = \sum_{j_1=-m}^m \sum_{j_2=-m}^m w[j_1, j_2] x[i_1 + j_1, i_2 + j_2]$$

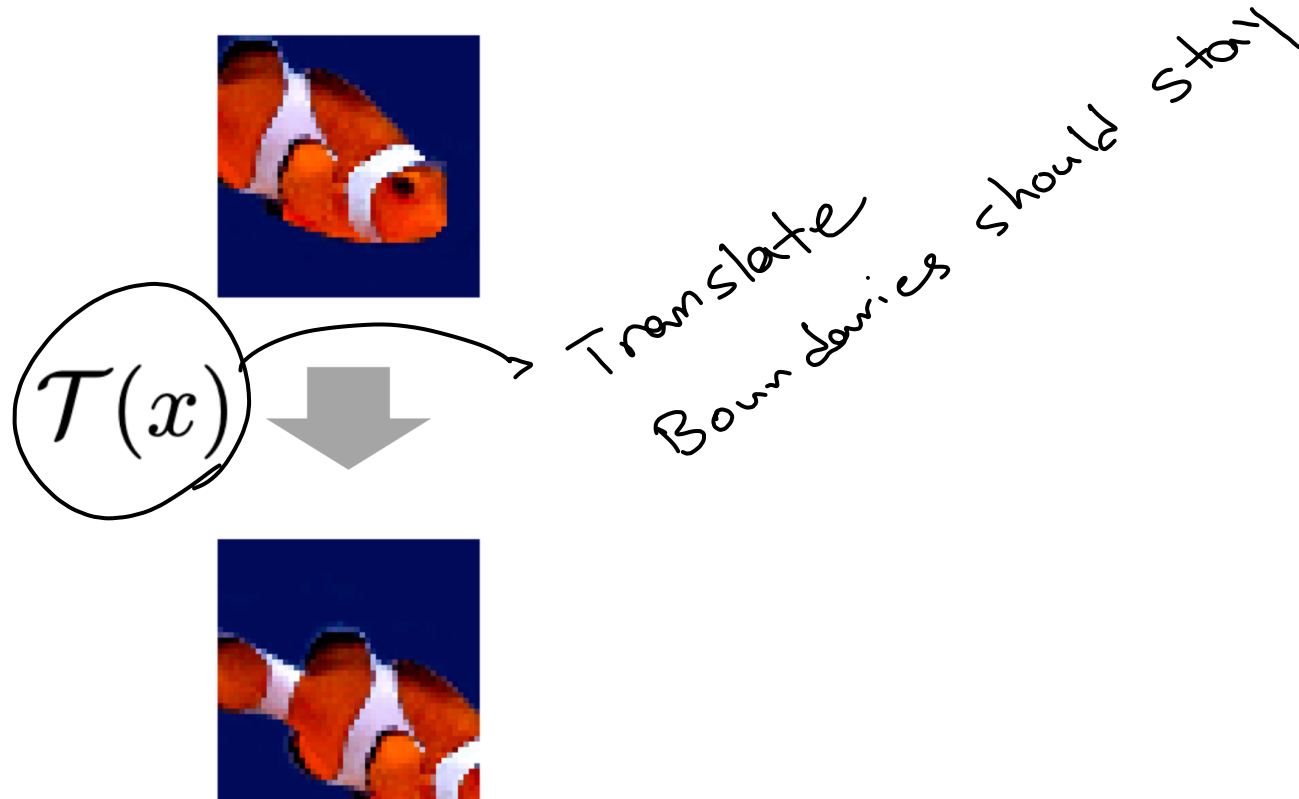
- Apply the same weights regardless of the window location

Translation-equivariance

- Equivariance: $f(\mathcal{T}(x)) = \mathcal{T}(f(x))$

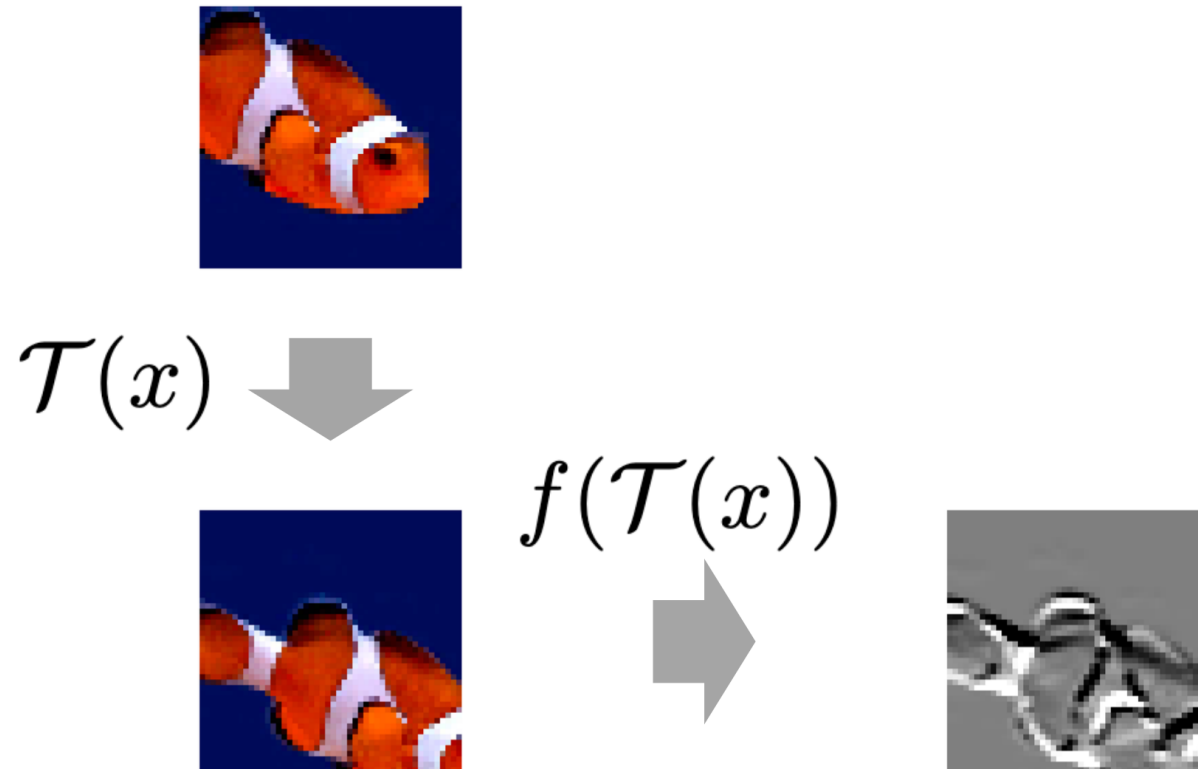
Translation-equivariance

- Equivariance: $f(\mathcal{T}(x)) = \mathcal{T}(f(x))$



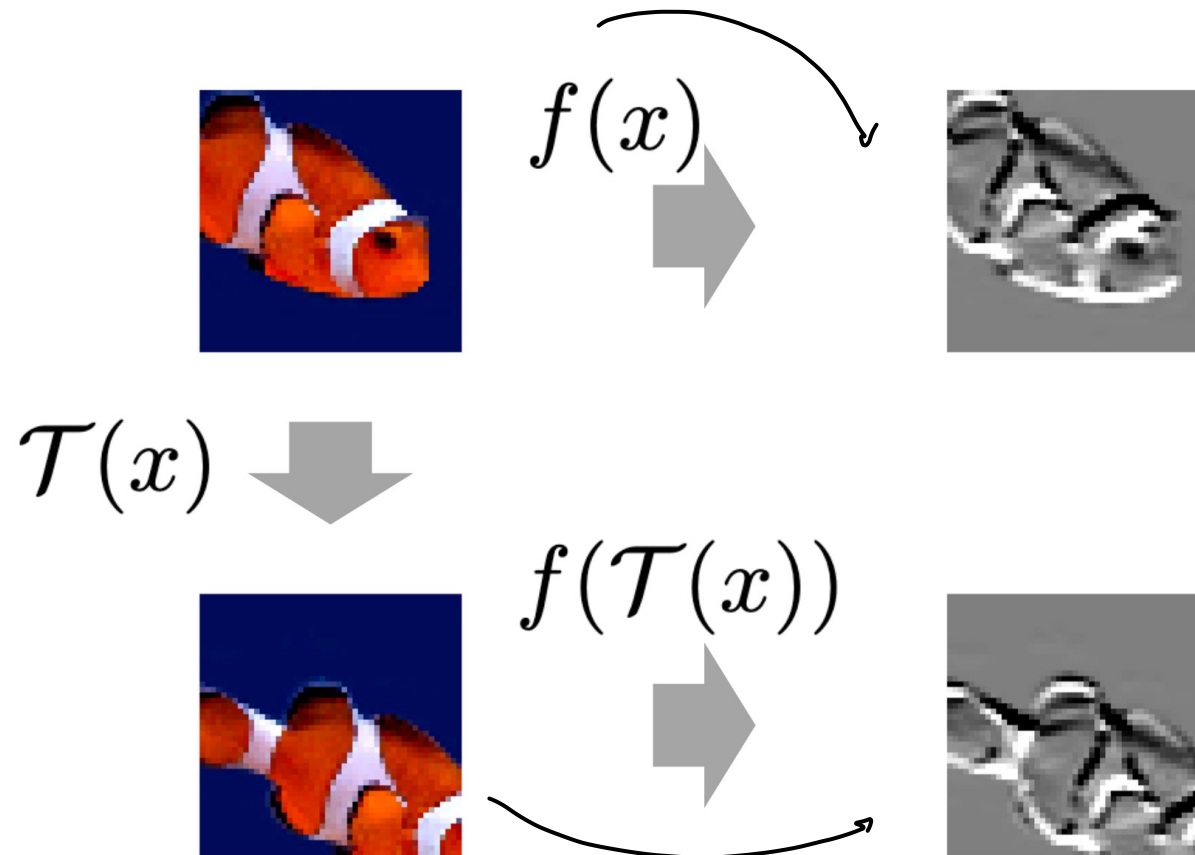
Translation-equivariance

- Equivariance: $f(\mathcal{T}(x)) = \mathcal{T}(f(x))$



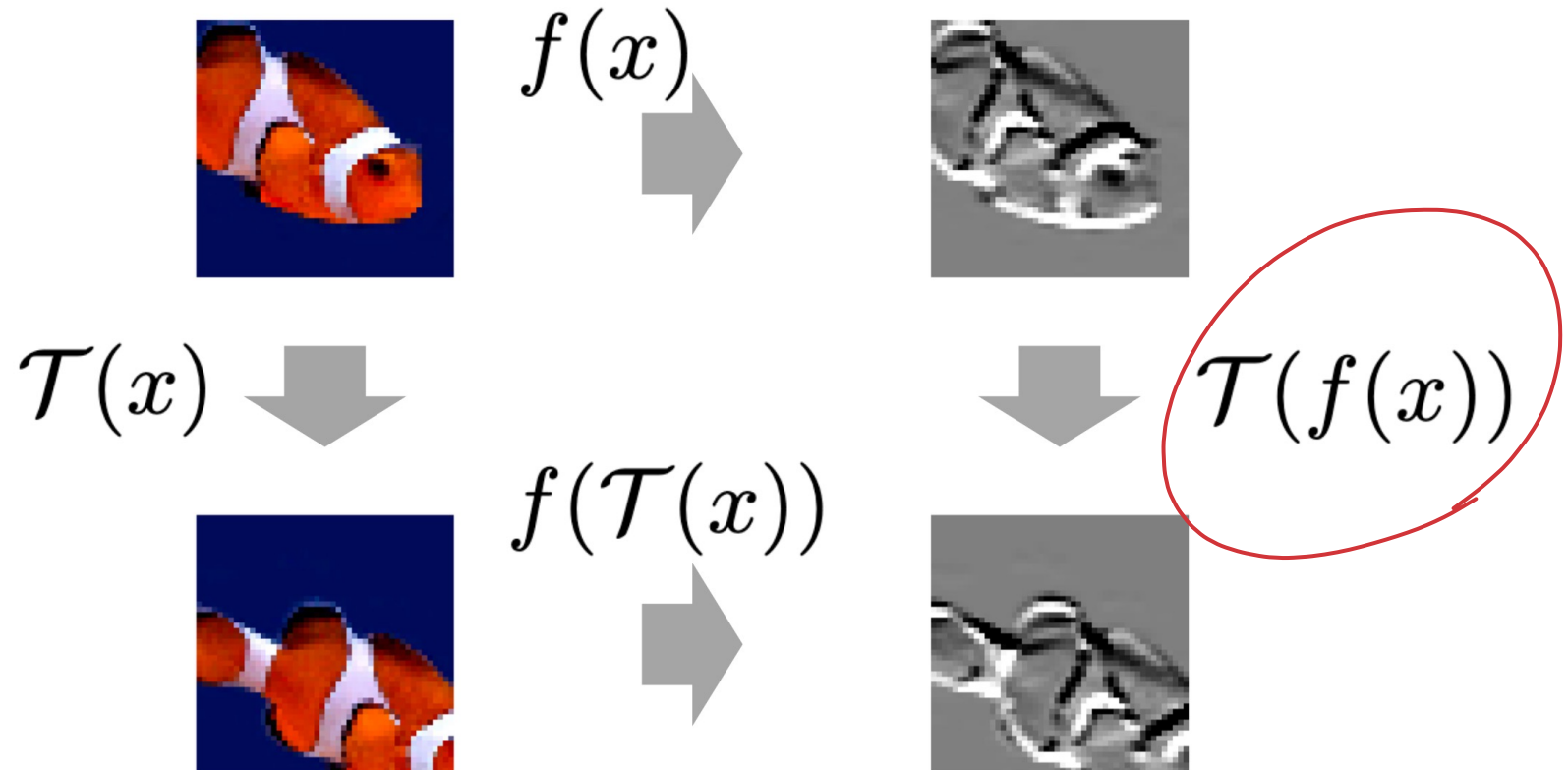
Translation-equivariance

- Equivariance: $f(\mathcal{T}(x)) = \mathcal{T}(f(x))$



Translation-equivariance

- Equivariance: $f(\mathcal{T}(x)) = \mathcal{T}(f(x))$



Assuming infinite size canvas

Local Connection + Weight Sharing

Let's plot it in a different way!

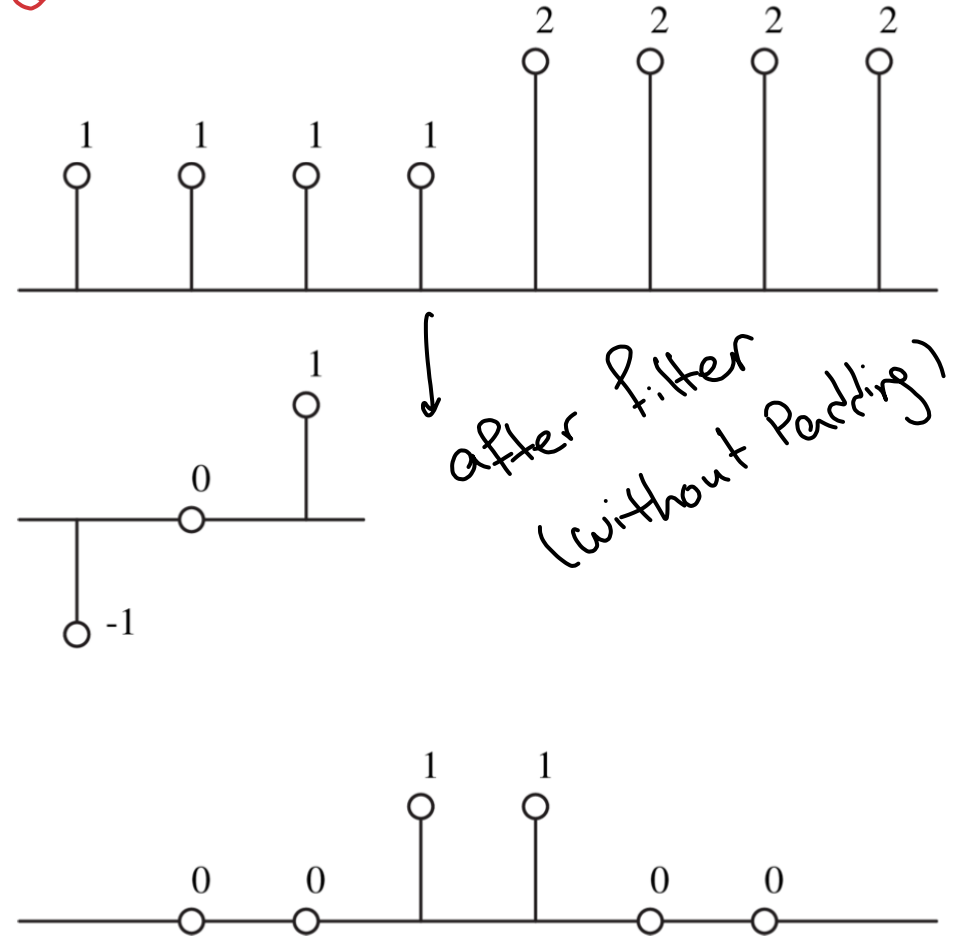


input

filter

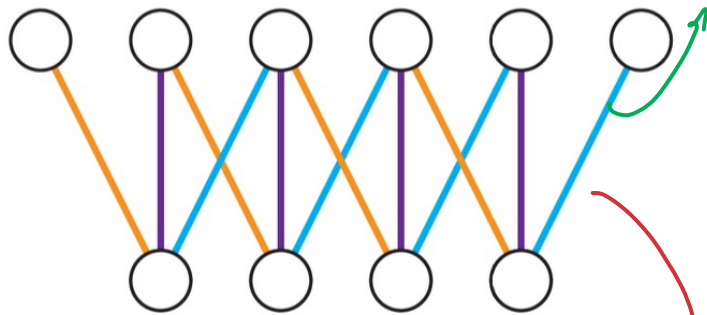
output

Not Fully Connected



Local Connection + Weight Sharing

convolution

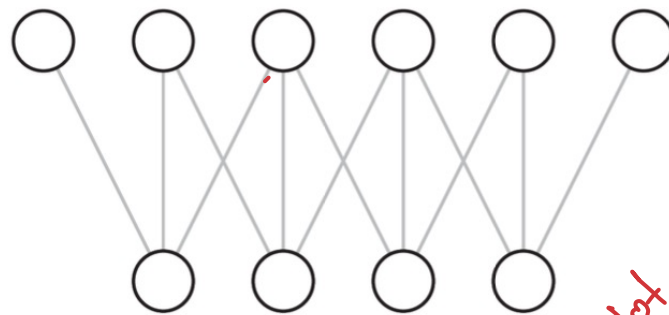


params = 3 here

They have the same weights

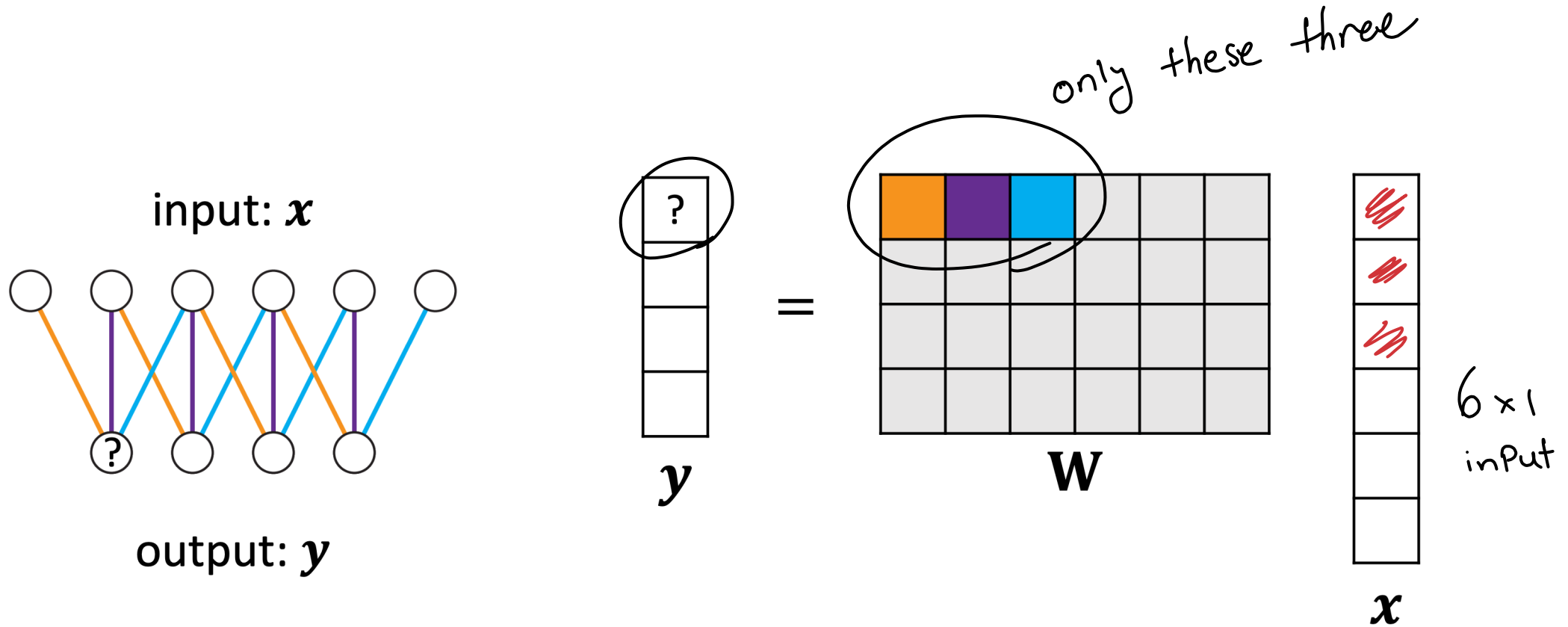
Colors are weights

locally connected

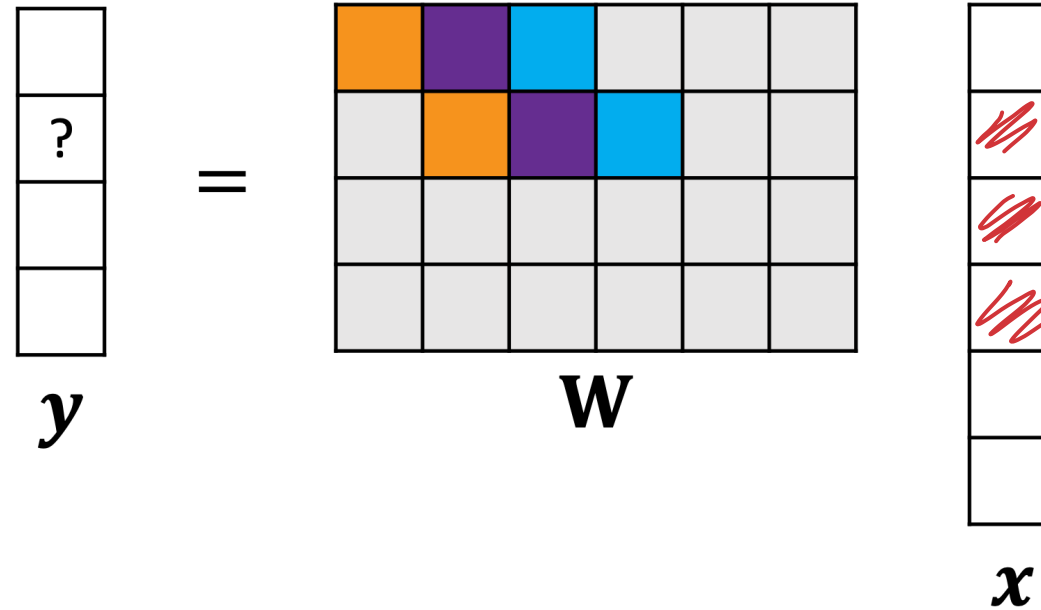
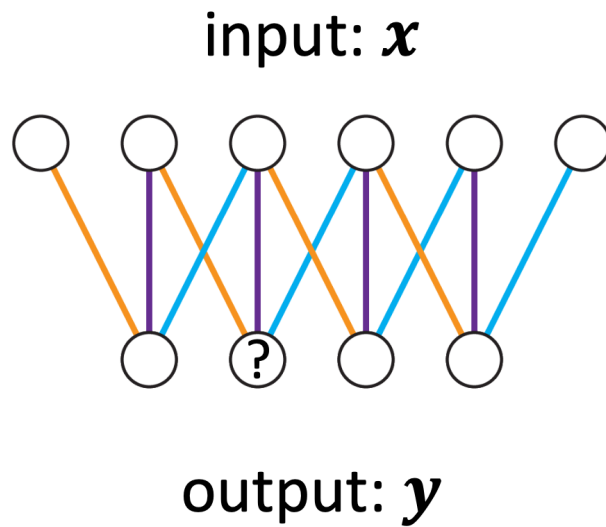


params = 12 here
no weight-sharing

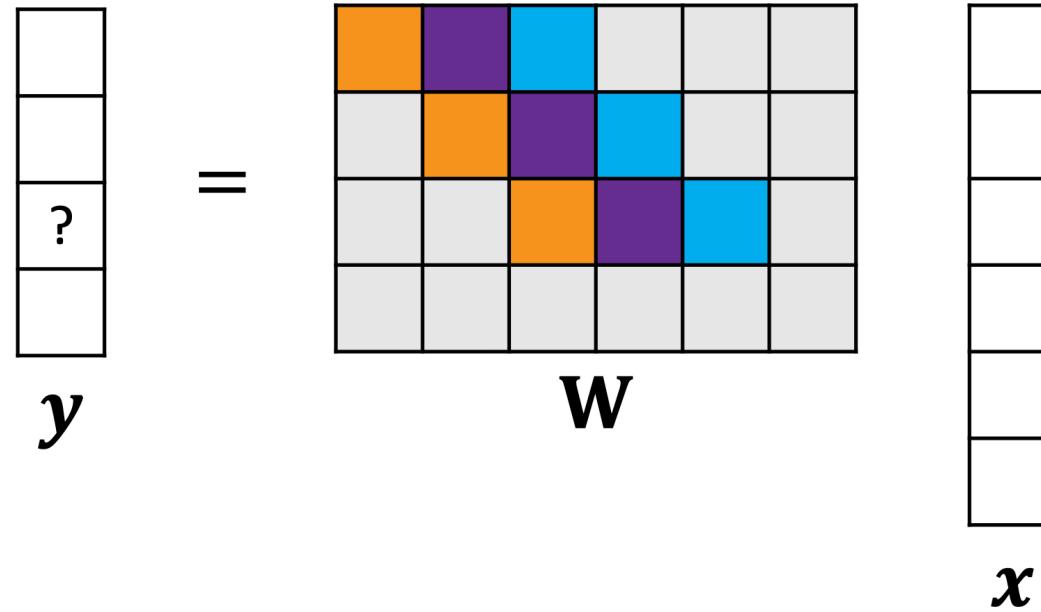
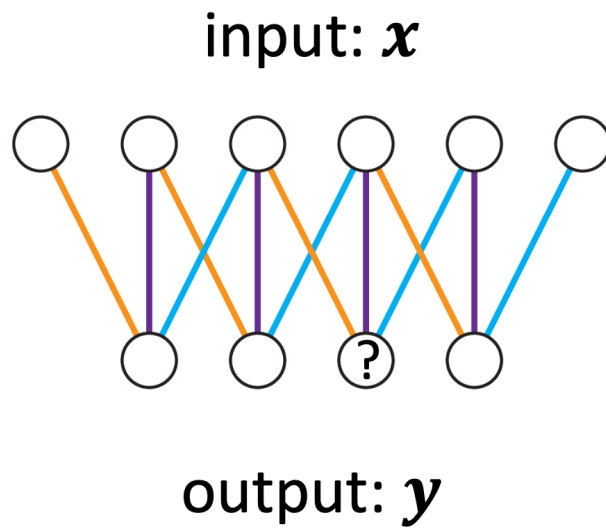
Convolution: Linear Transformation



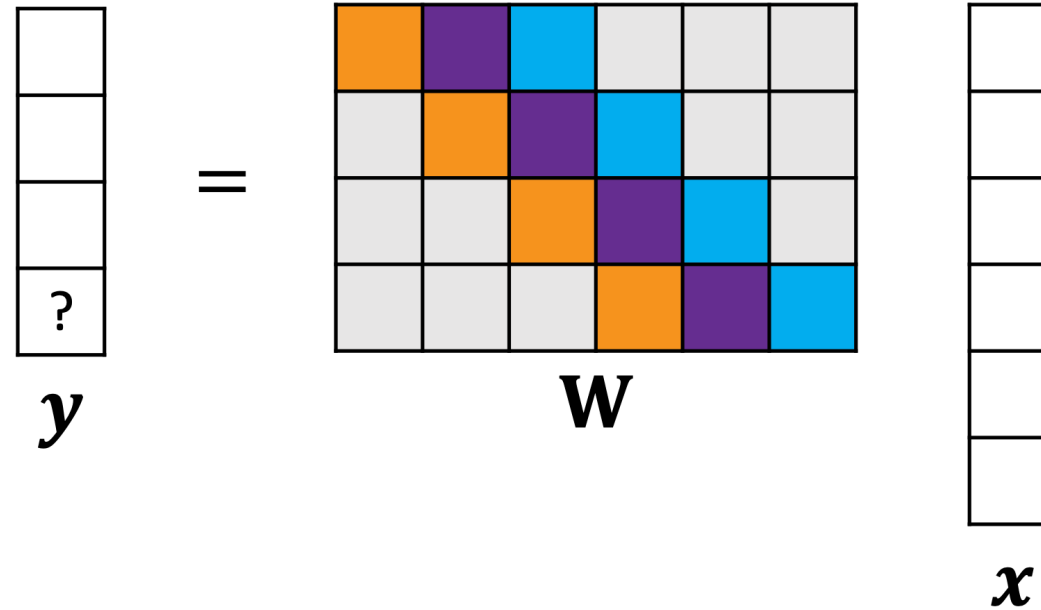
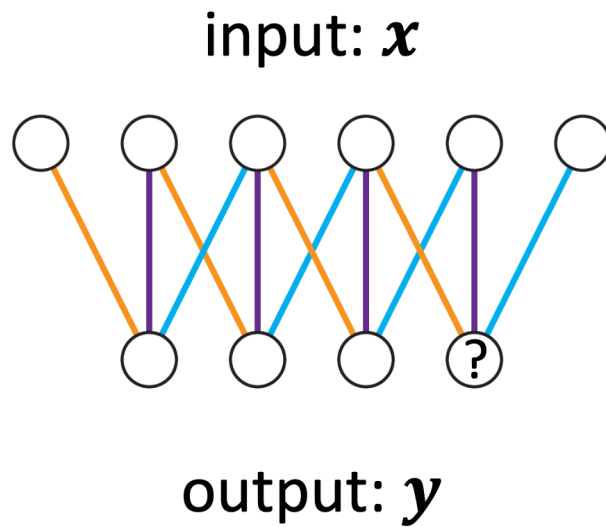
Convolution: Linear Transformation



Convolution: Linear Transformation

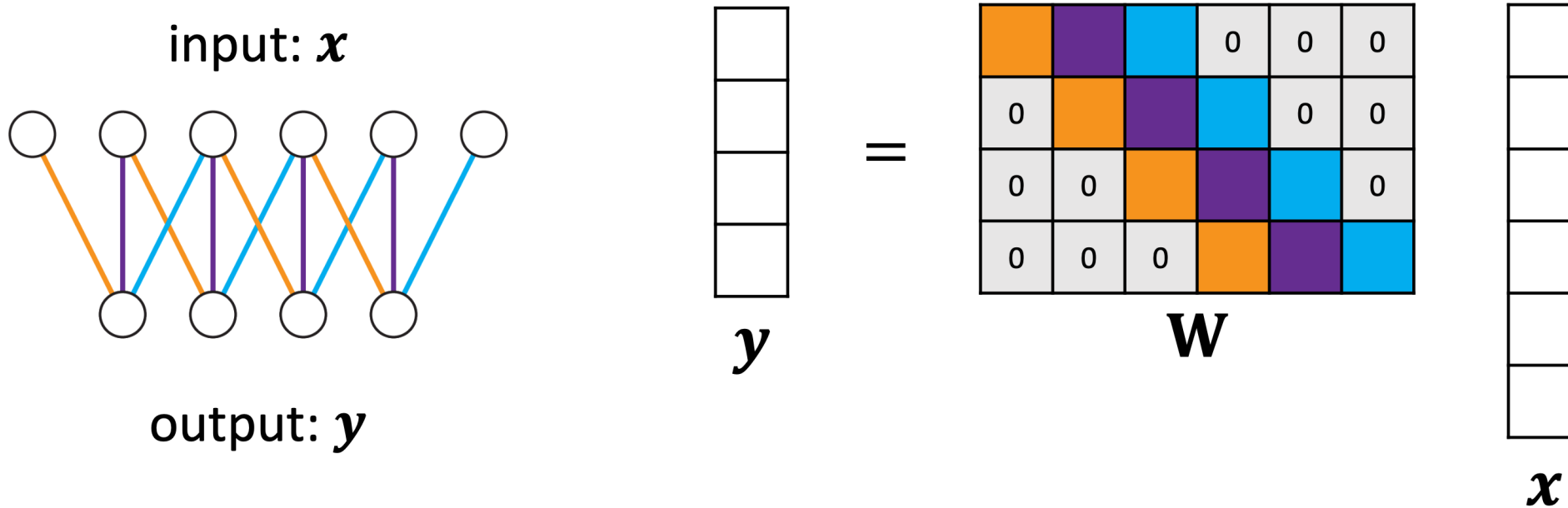


Convolution: Linear Transformation



Convolution: Linear Transformation

↪ it is a linear transformation.

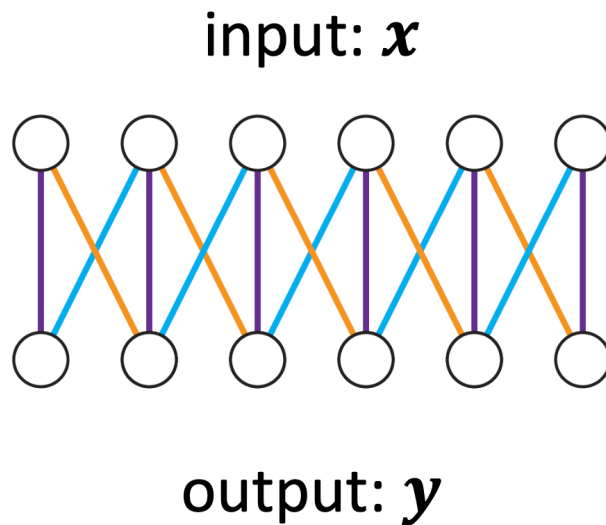


- **local connection:** W is sparse
- **weight-sharing:** # params = 3 here in W

↪ Number of Parameters

Convolution: Linear Transformation

↓
is very low

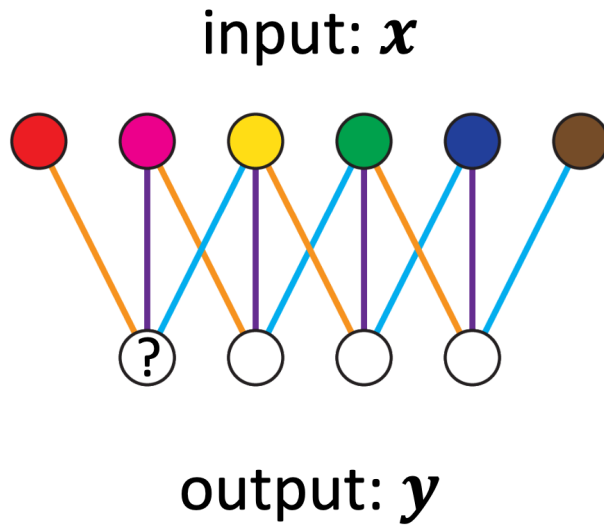


$$\mathbf{y} = \mathbf{W} \mathbf{x}$$

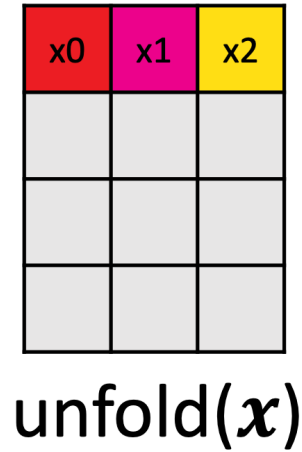
$\mathbf{y}$ $\mathbf{W}$ $\mathbf{x}$

- This is the case with padding
- *Exercise:* what about stride = 2?
- *Exercise:* what about 2-D convolution?

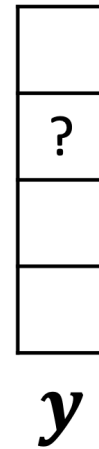
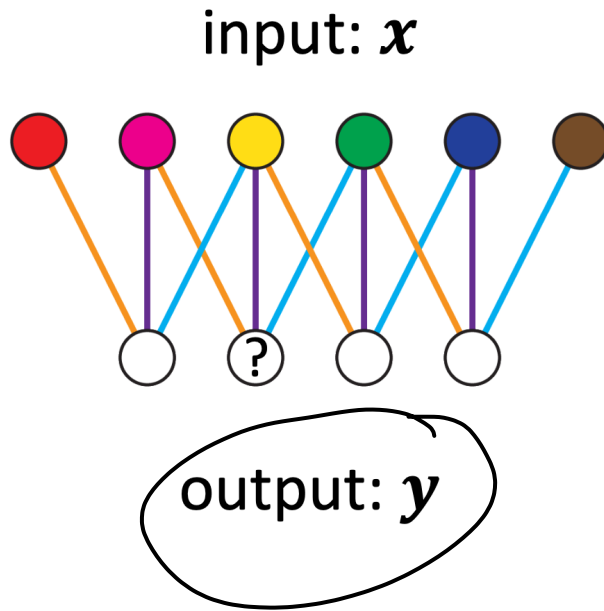
Convolution: unfold/im2col



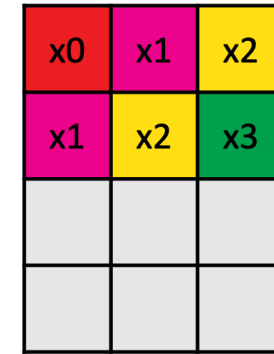
=



Convolution: unfold/im2col

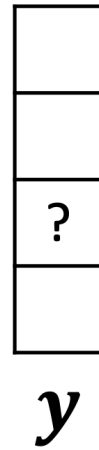
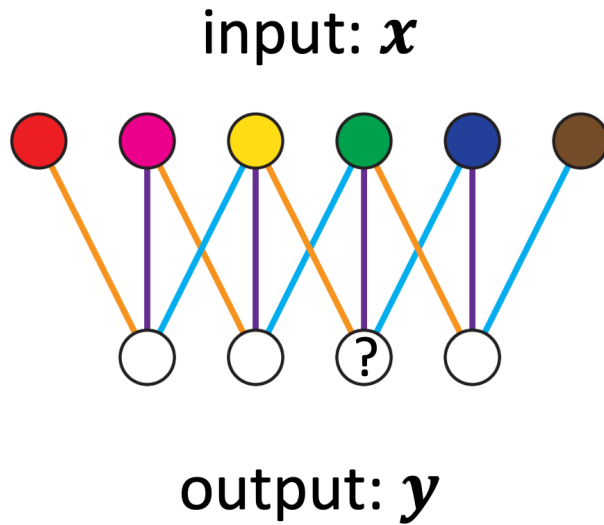


=

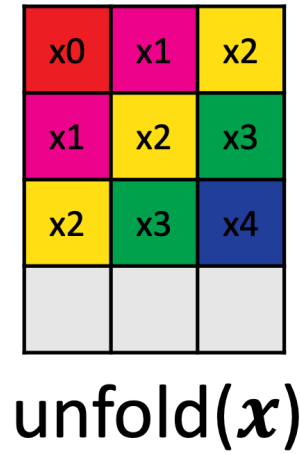


it also reduce the size of x .

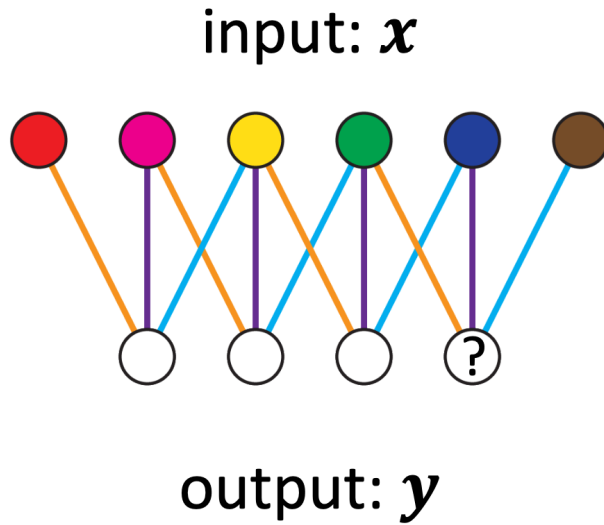
Convolution: unfold/im2col



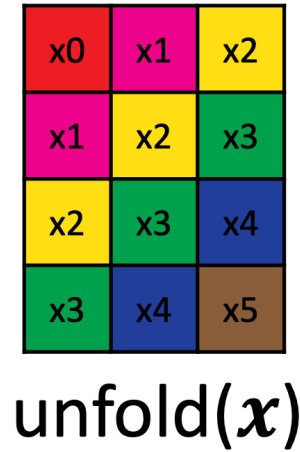
=



Convolution: unfold/im2col



=



Convolution: unfold/im2col

↓
Images to Column

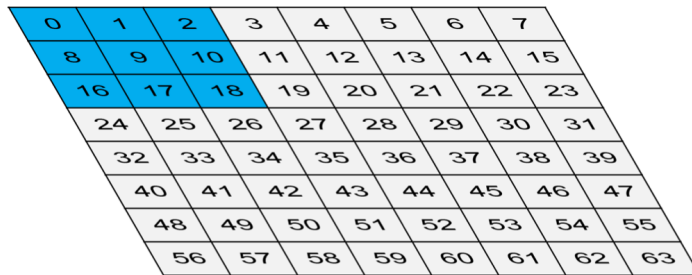
0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31
32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47
48	49	50	51	52	53	54	55
56	57	58	59	60	61	62	63

2D im

0									
1									
2									
8									
9									
10									
16									
17									
18									

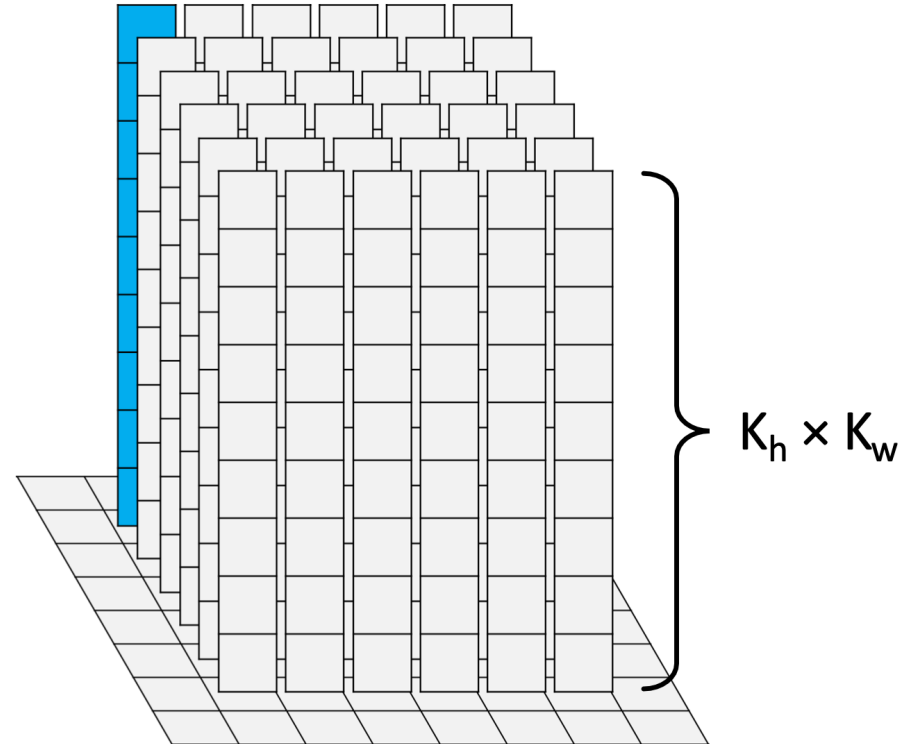
im2col

Convolution: unfold/im2col



0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31
32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47
48	49	50	51	52	53	54	55
56	57	58	59	60	61	62	63

2D im



im2col

Convolution: unfold/im2col

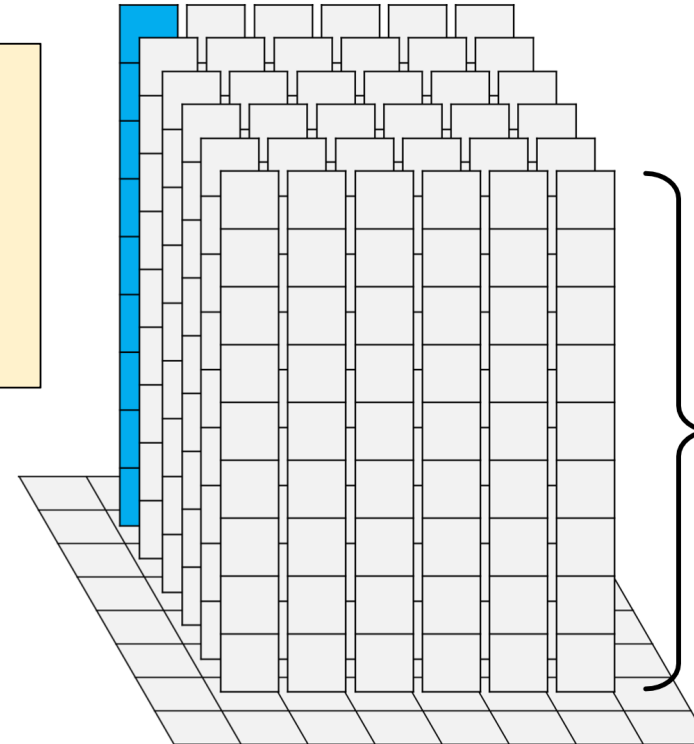
Reducing the size

im2col

- gather local information
- flatten into a "col" ($K_h \times K_w$ channels)
- followed by 1×1 conv

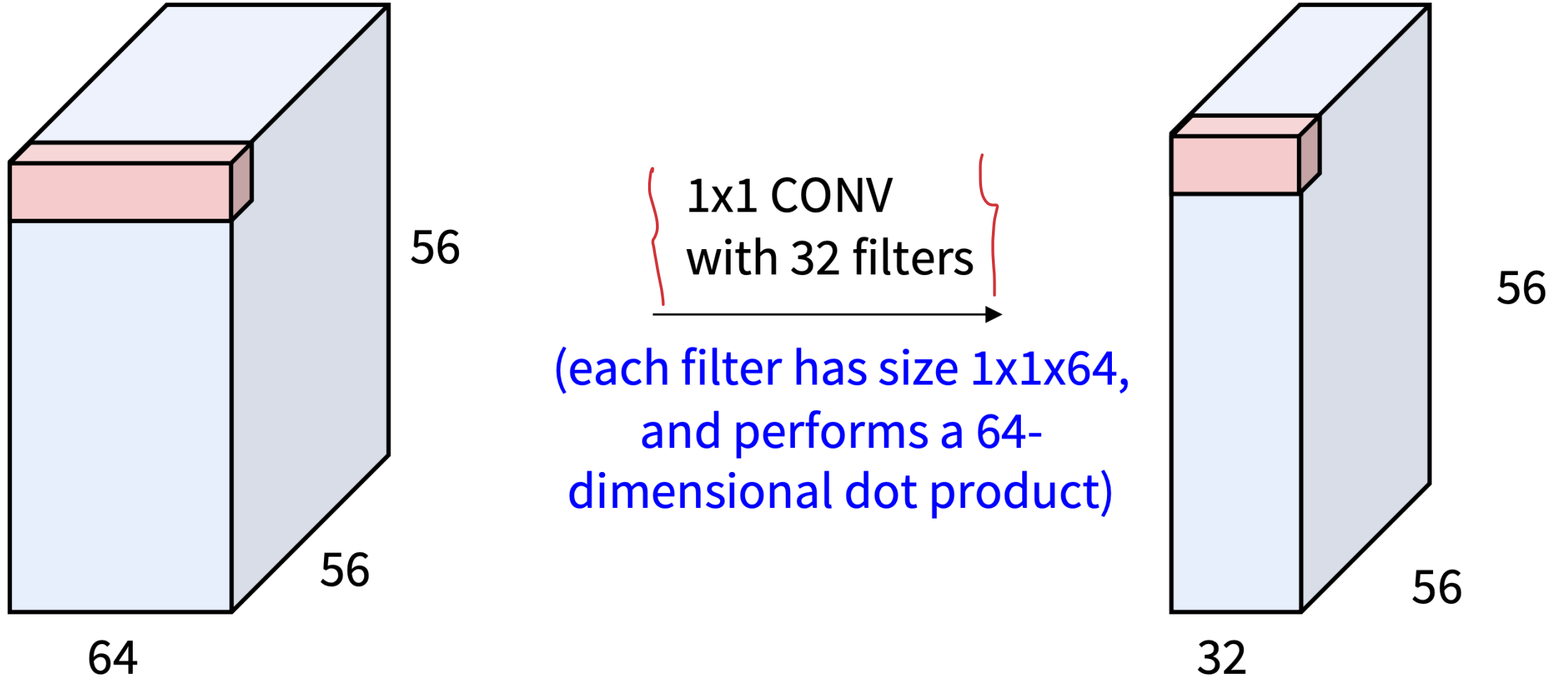
0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31
32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47
48	49	50	51	52	53	54	55
56	57	58	59	60	61	62	63

2D im

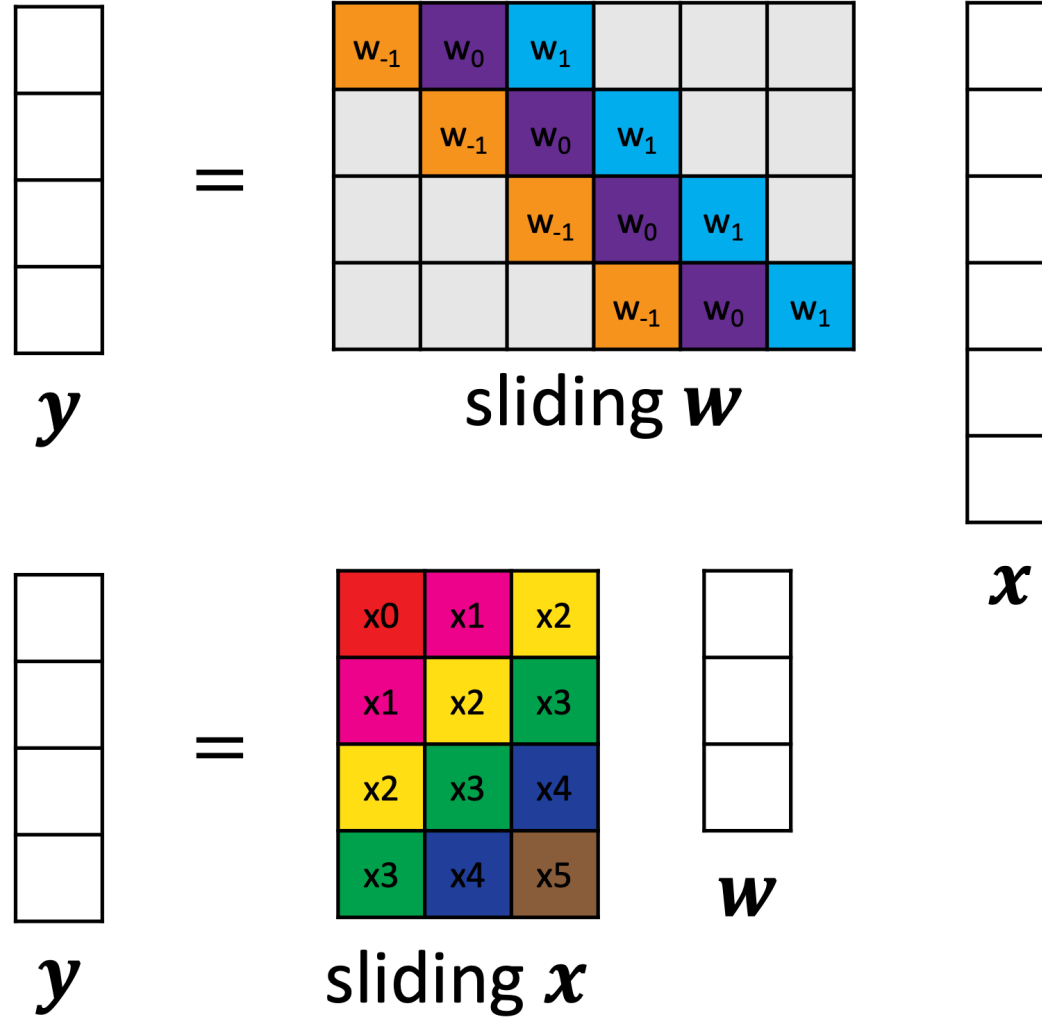
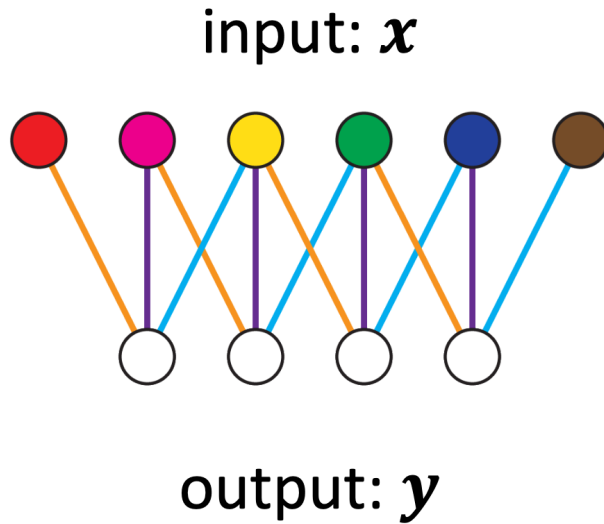


im2col

1x1 Convolution



Convolution: unfold/im2col



im2cols

Next Topic: Convolutional Neural Networks

Motivation: X-Ray Abnormality Detection

- Want to build a system that recognizes abnormalities in x-rays:



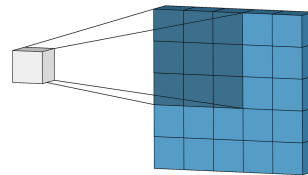
“Abnormality detected”
(binary classification)

for images

- Applications:
 - Fast detection of tuberculosis, pneumonia, lung cancer, and so on.
- Deep learning has led to incredible progress on computer vision tasks.
 - Much of this progress has been driven by convolutional neural networks (CNNs).

Convolutional Neural Network (CNN) Motivation

- Consider training neural networks on 500 pixel by 500 pixel images.
 - So the number of inputs 'd' to first layer is 250,000 inputs. *let's make that + 10,000*
- If first layer has $k=10,000$, then 'W' has 2.5 billion parameters. *Too much*
 - We want to avoid this huge number (due to storage and overfitting).
- CNNs drastically reduce the number of parameters.
 - Main way they do this is using layers that look like convolutions:



Filters

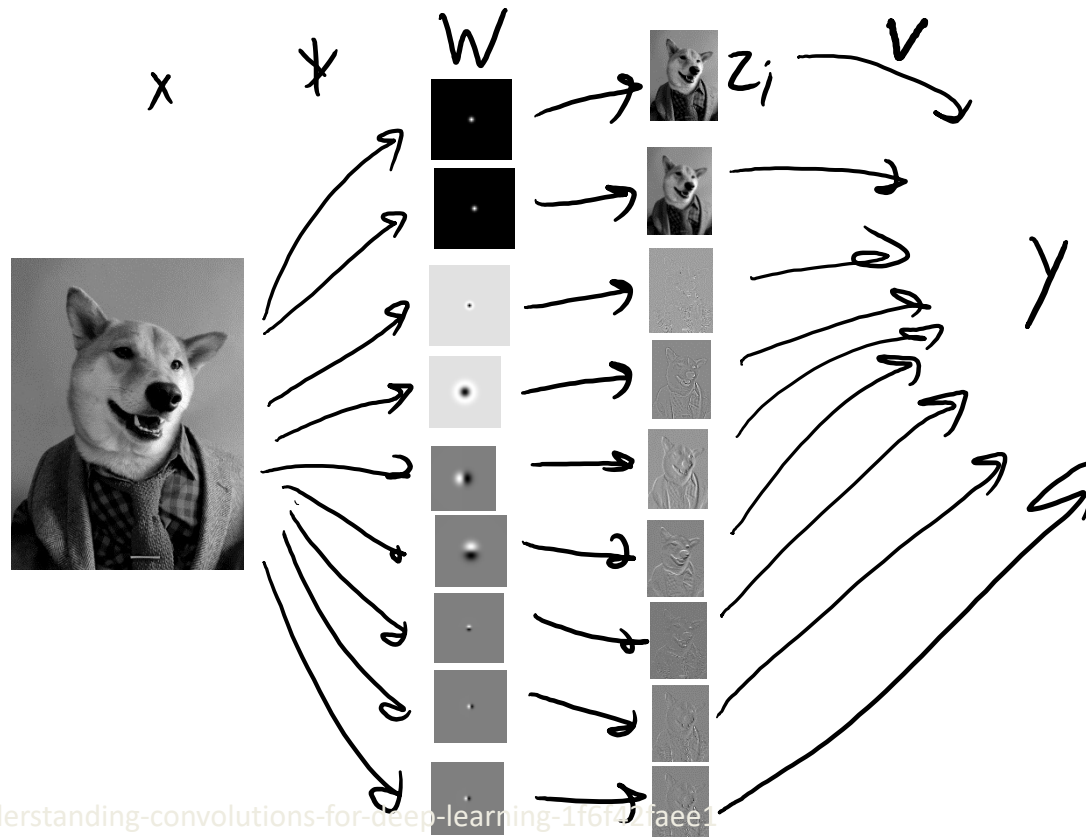
$$\begin{array}{ccc} 250,000 & \longrightarrow & 10,000 \\ & \downarrow & \\ & w = 10,000 & \\ & \times & \\ & 250,000 & \end{array}$$

- Hidden units only depend on a small number of inputs.
- Using the same parameters on the connections of many activations.

Motivation for Convolutional Neural Networks

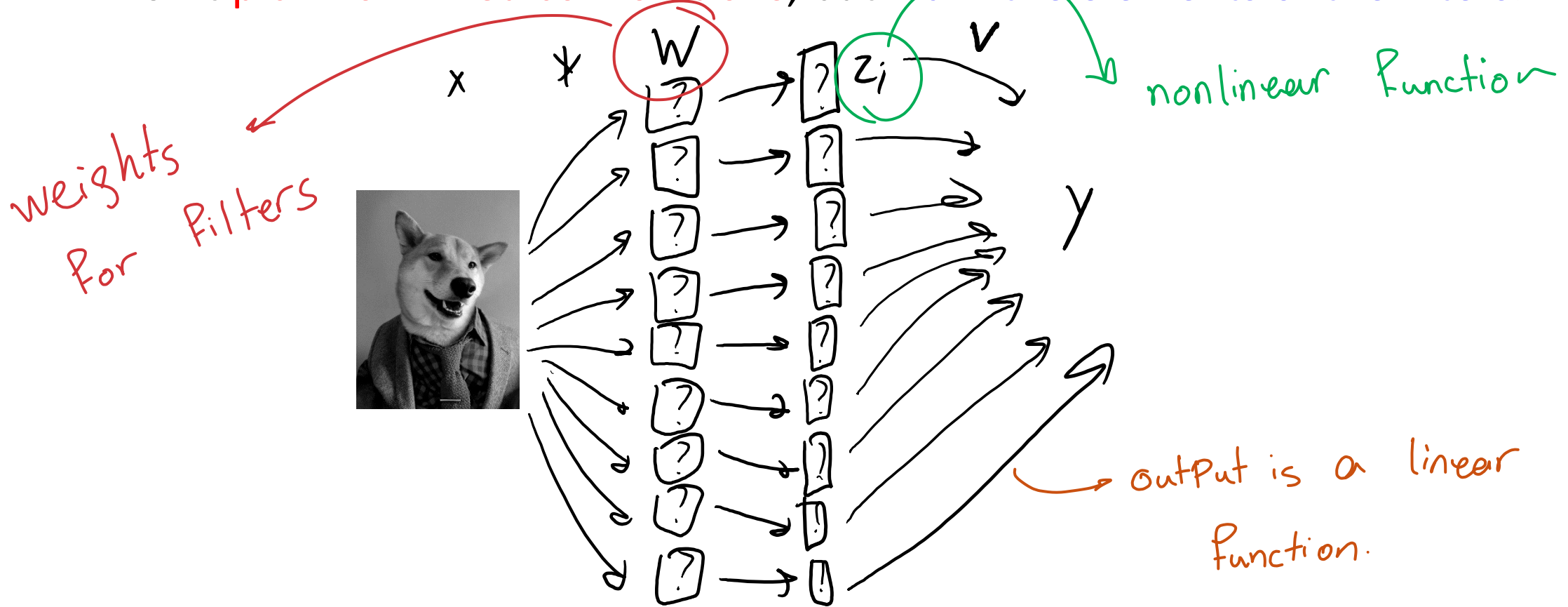
2.5 billion
Parameters

- Classic vision methods uses **fixed convolutions** as features:
 - Usually have **different types/variances/orientations**.
 - Can do subsampling or take **maxes across locations/orientations/scales**.



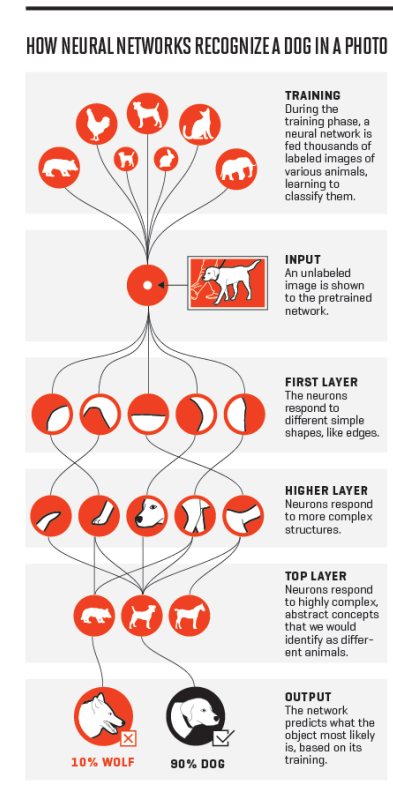
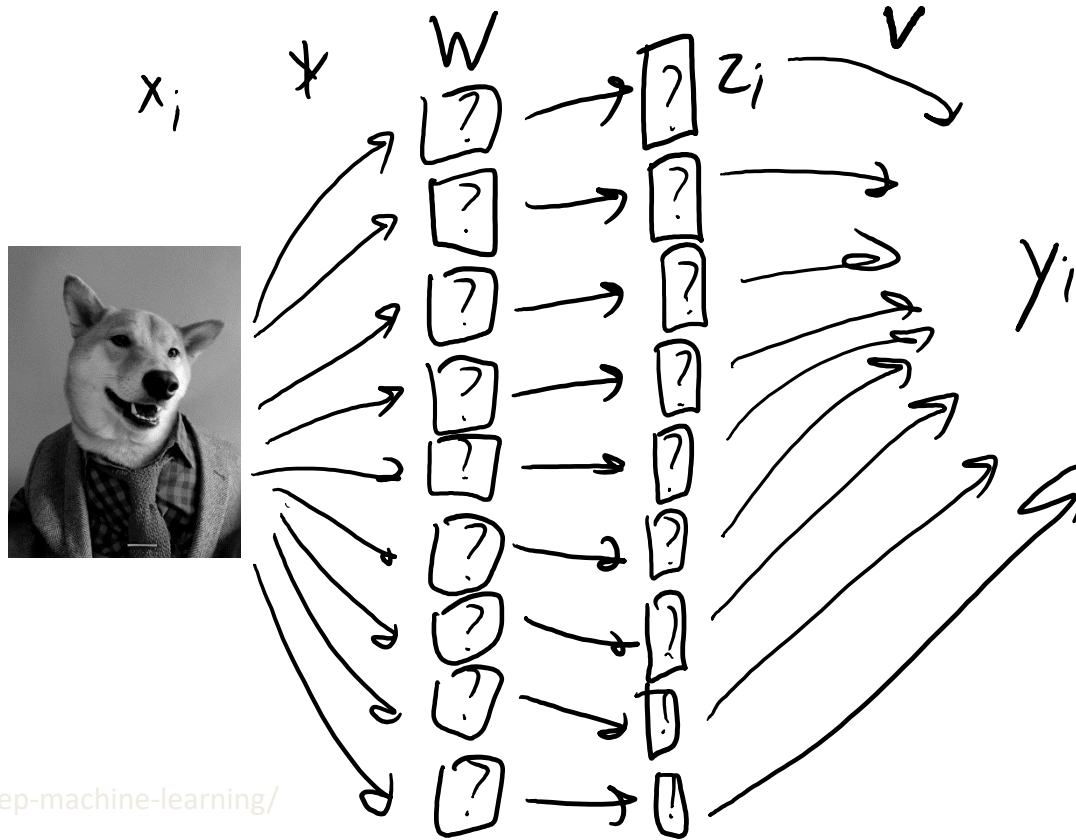
Motivation for Convolutional Neural Networks

- Convolutional neural networks learn the convolutions:
 - Learning 'W' and 'v' automatically chooses types/variances/orientations.
 - Don't pick from fixed convolutions, but learn the elements of the filters.



Motivation for Convolutional Neural Networks

- Convolutional neural networks learn the convolutions:
 - Learning 'W' and 'v' automatically chooses types/variances/orientations.
 - Can do multiple layers of convolution to get deep hierarchical features.



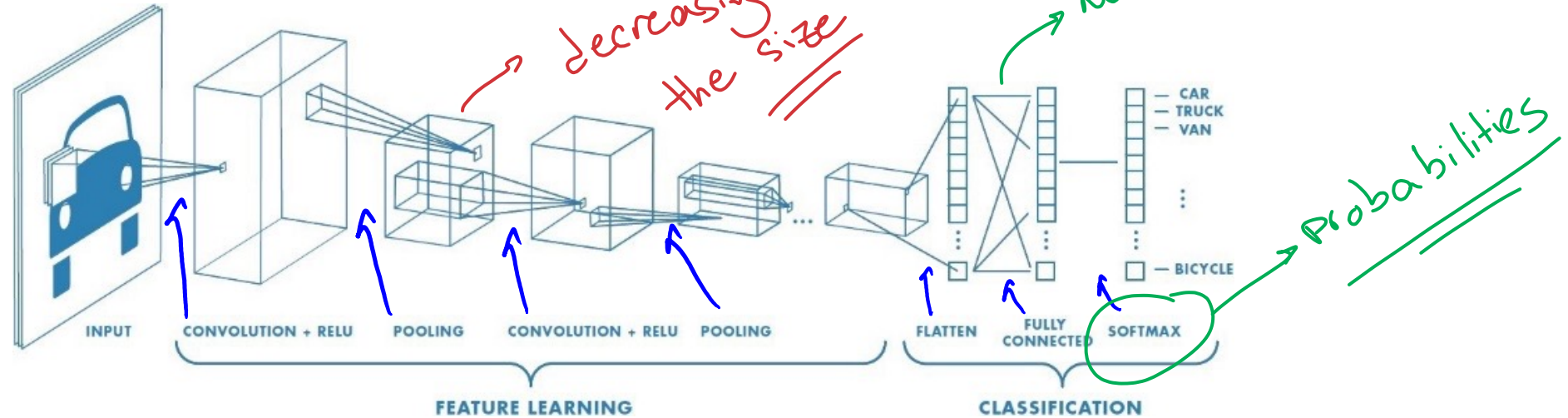
Convolutional Neural Network (ConvNet, CNN)

- Core idea of deep learning:
Composing basic operations into complex functions
- Common operations of ConvNet
 - convolution
 - activation function
 - pooling
 - fully-connected layer
 - softmax
- **Train with BackProp!**

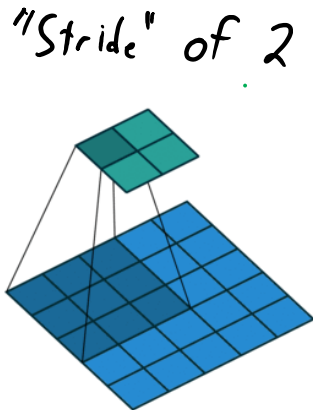


Convolutional Neural Networks

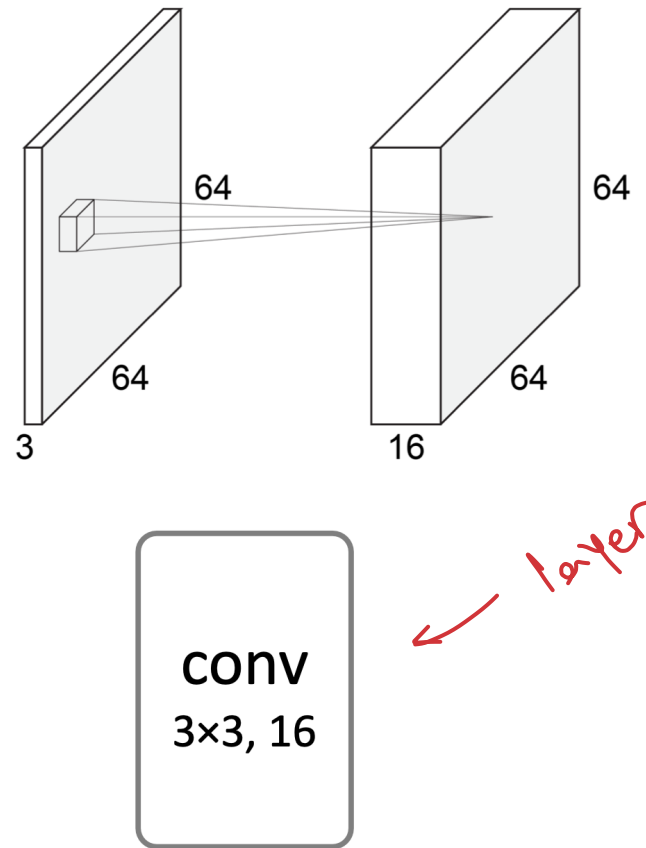
- Classic **architecture** of a convolutional neural network:



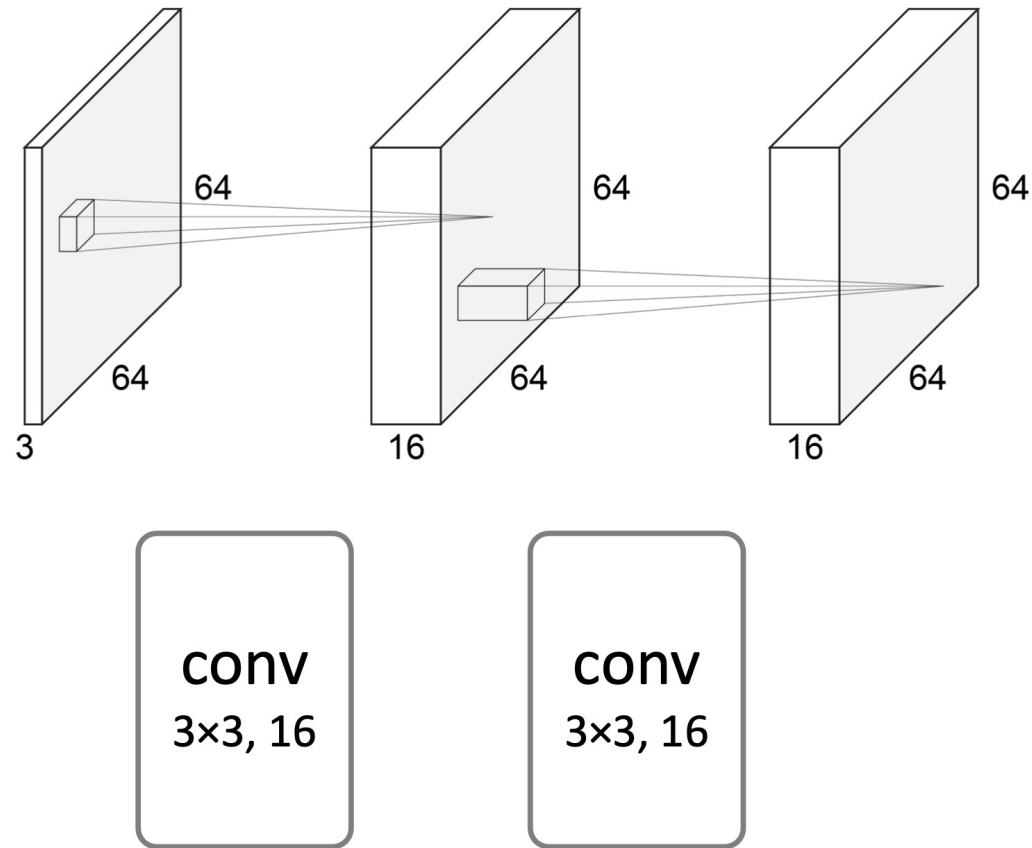
- Convolution layers:**
 - Apply convolution with several different filters.
 - Sometimes these have a "stride": skip several pixels between applying filter.
- Pooling layers:**
 - Aggregate regions to create smaller images (usually "max pooling").
- Fully-connected layers:** usual "multiplication by W " in layer.



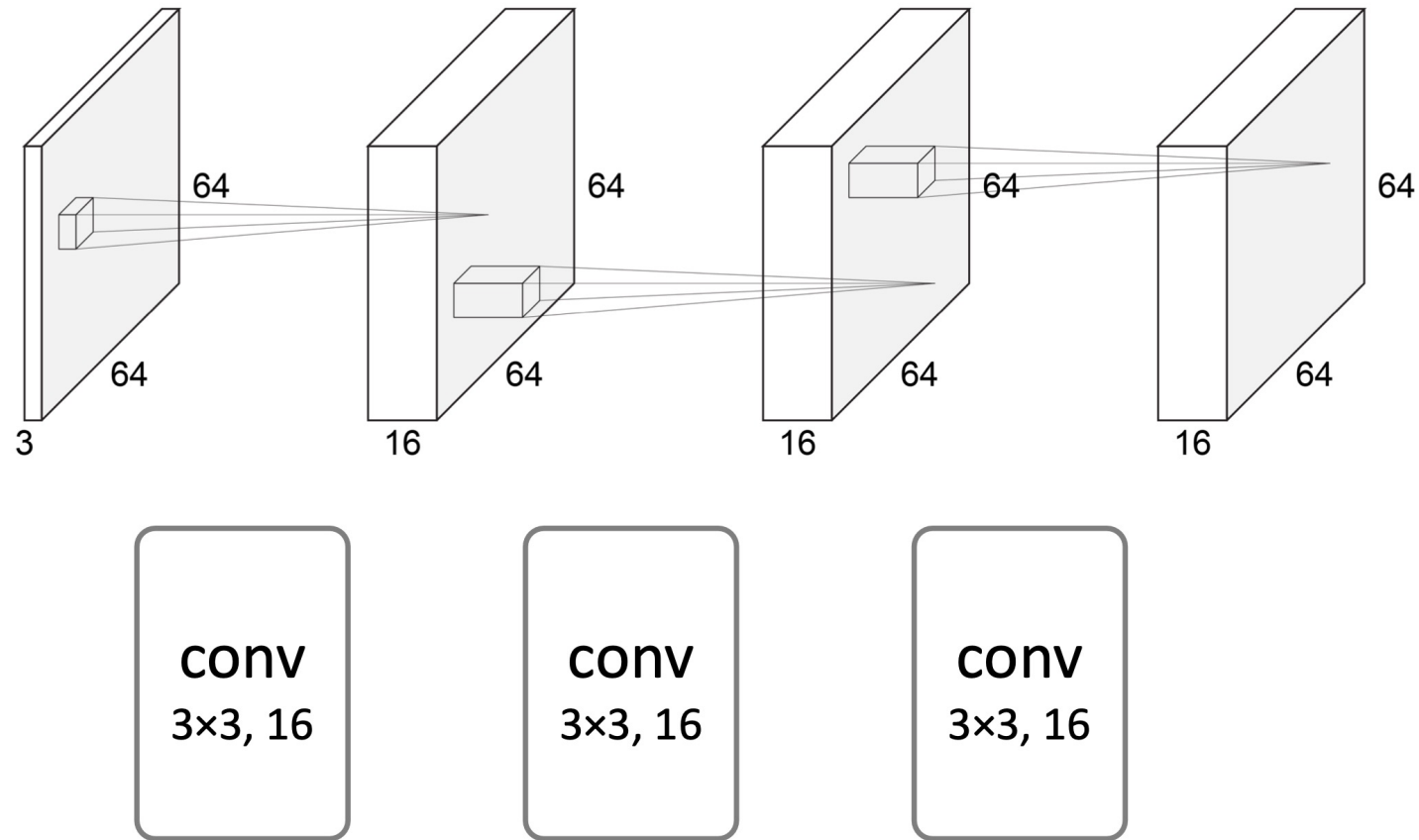
Compositions of Convolution Layers



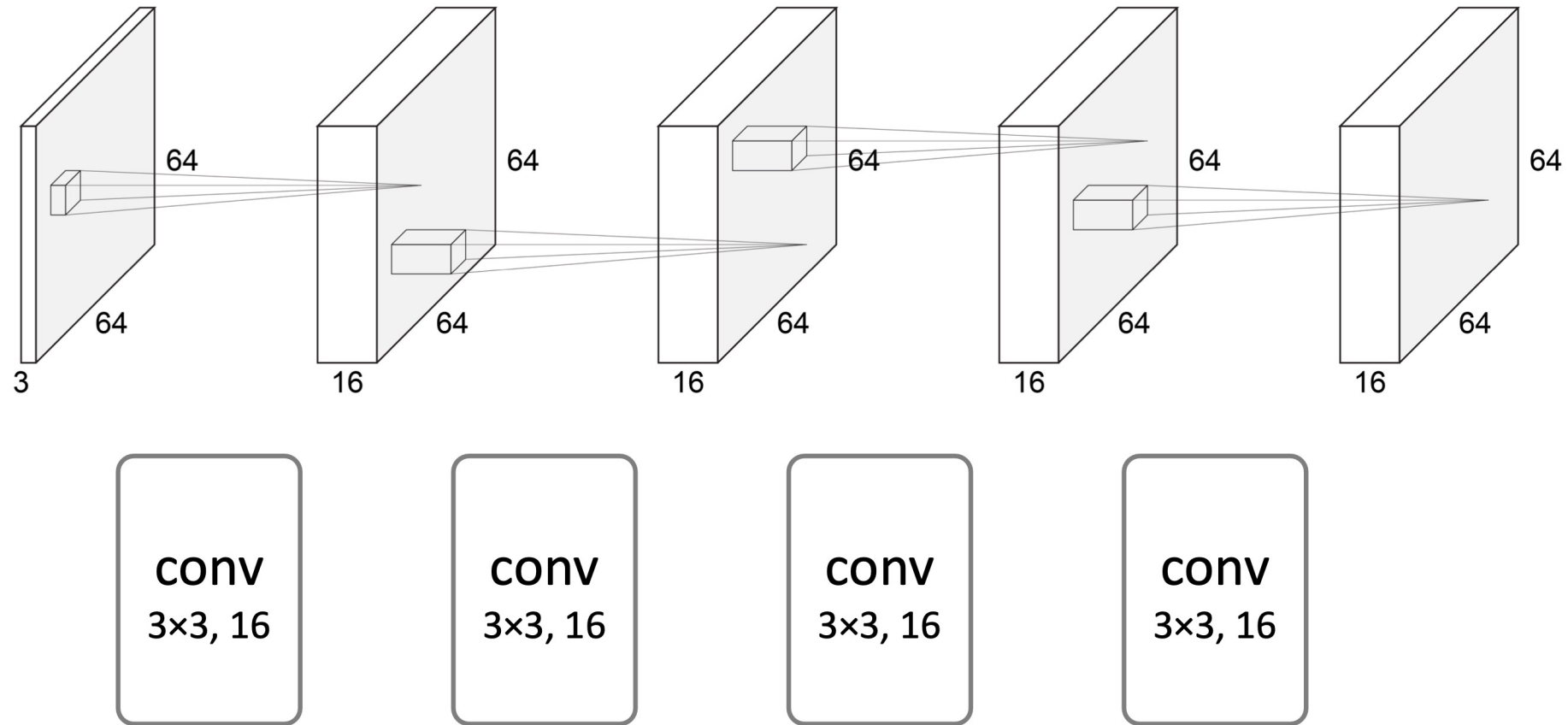
Compositions of Convolution Layers



Compositions of Convolution Layers

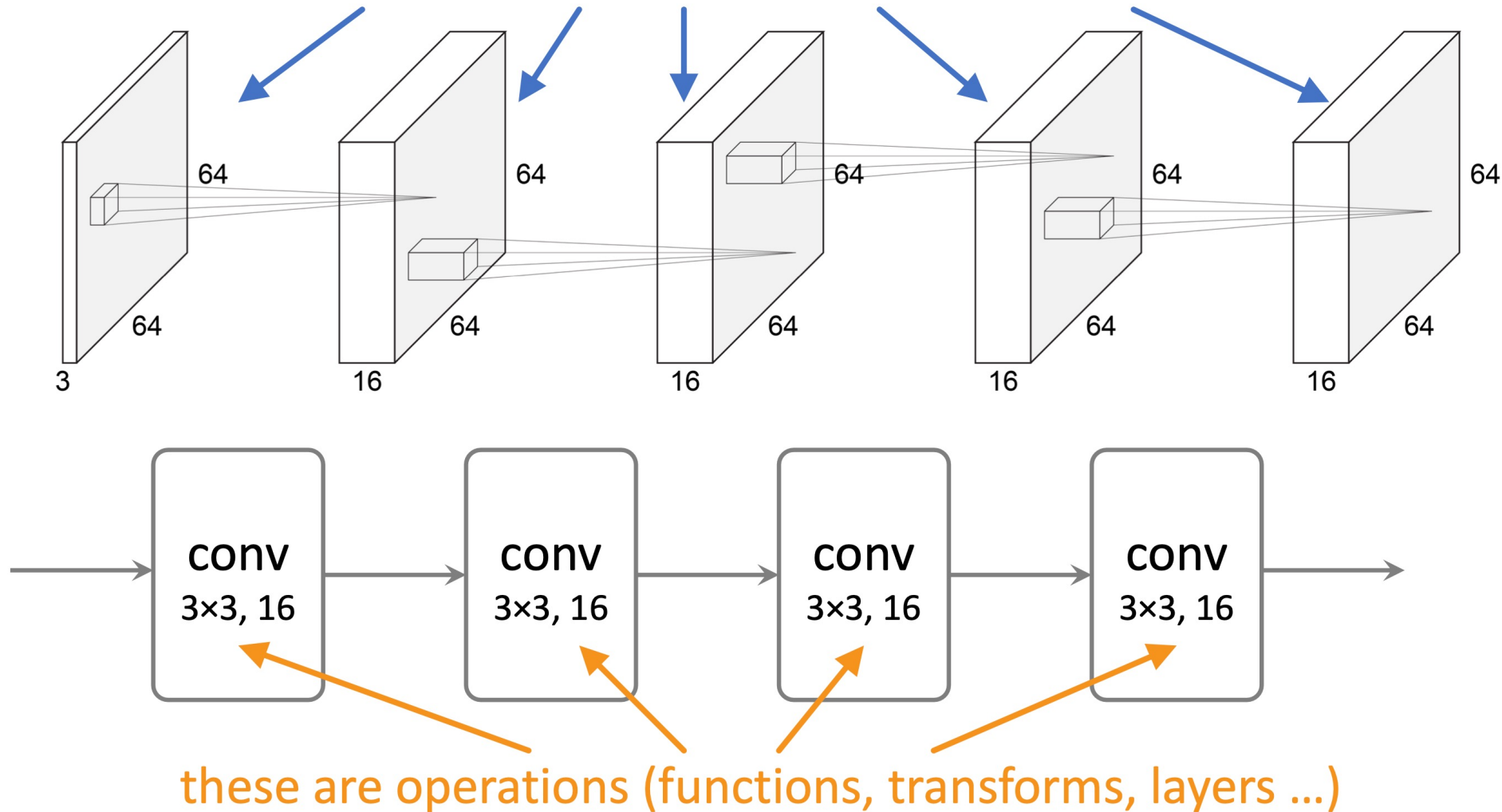


Compositions of Convolution Layers



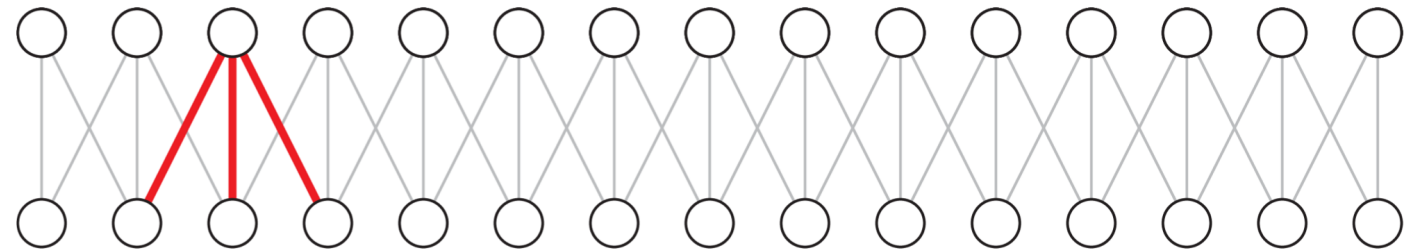
Compositions of Convolution Layers

these are activations (features, embeddings, tensors ...)



Receptive Fields

kernel = 3

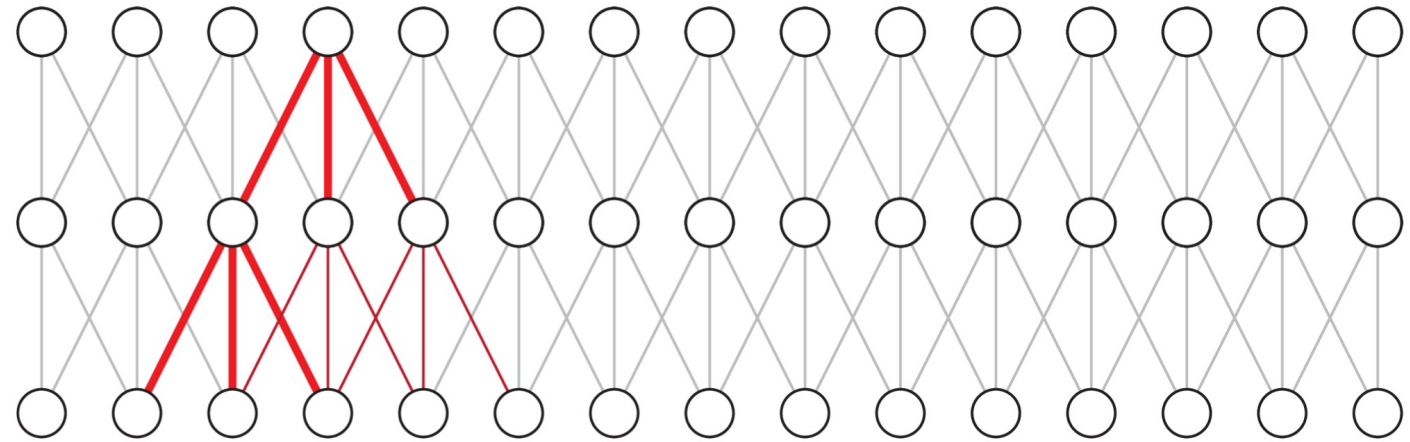


receptive field = 3

Receptive Fields

kernel = 3

kernel = 3



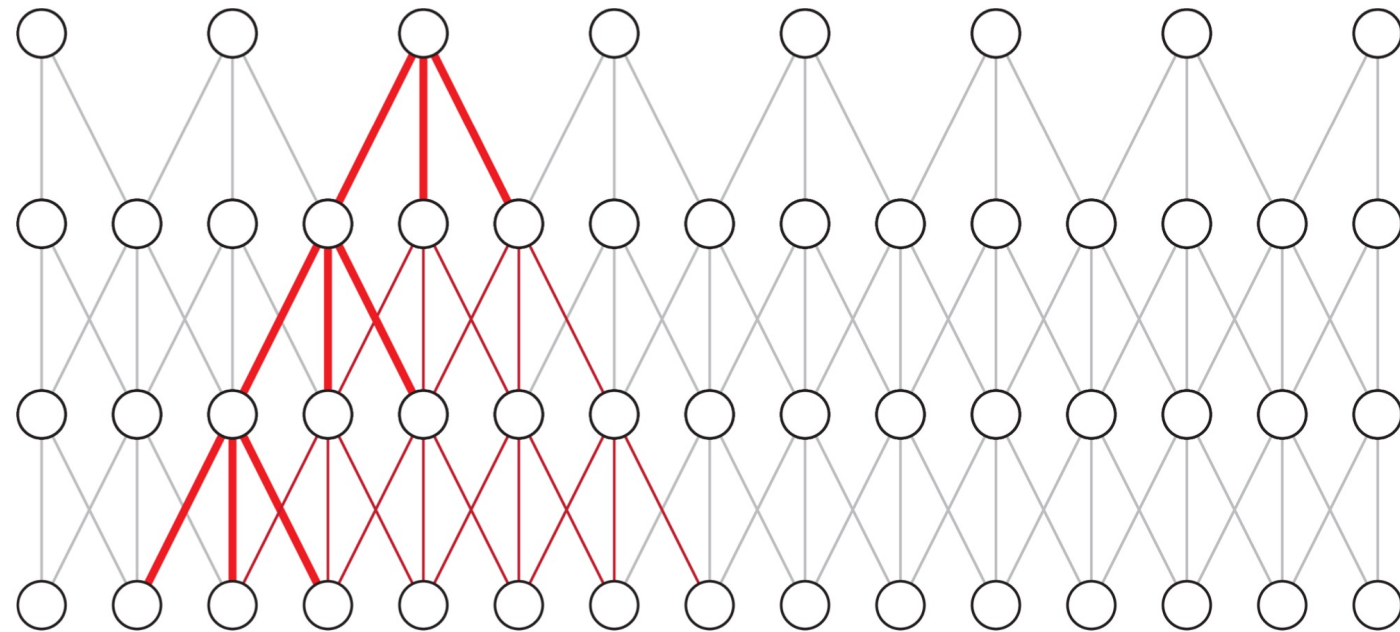
receptive field = 5

Receptive Fields

kernel = 3, str = 2

kernel = 3

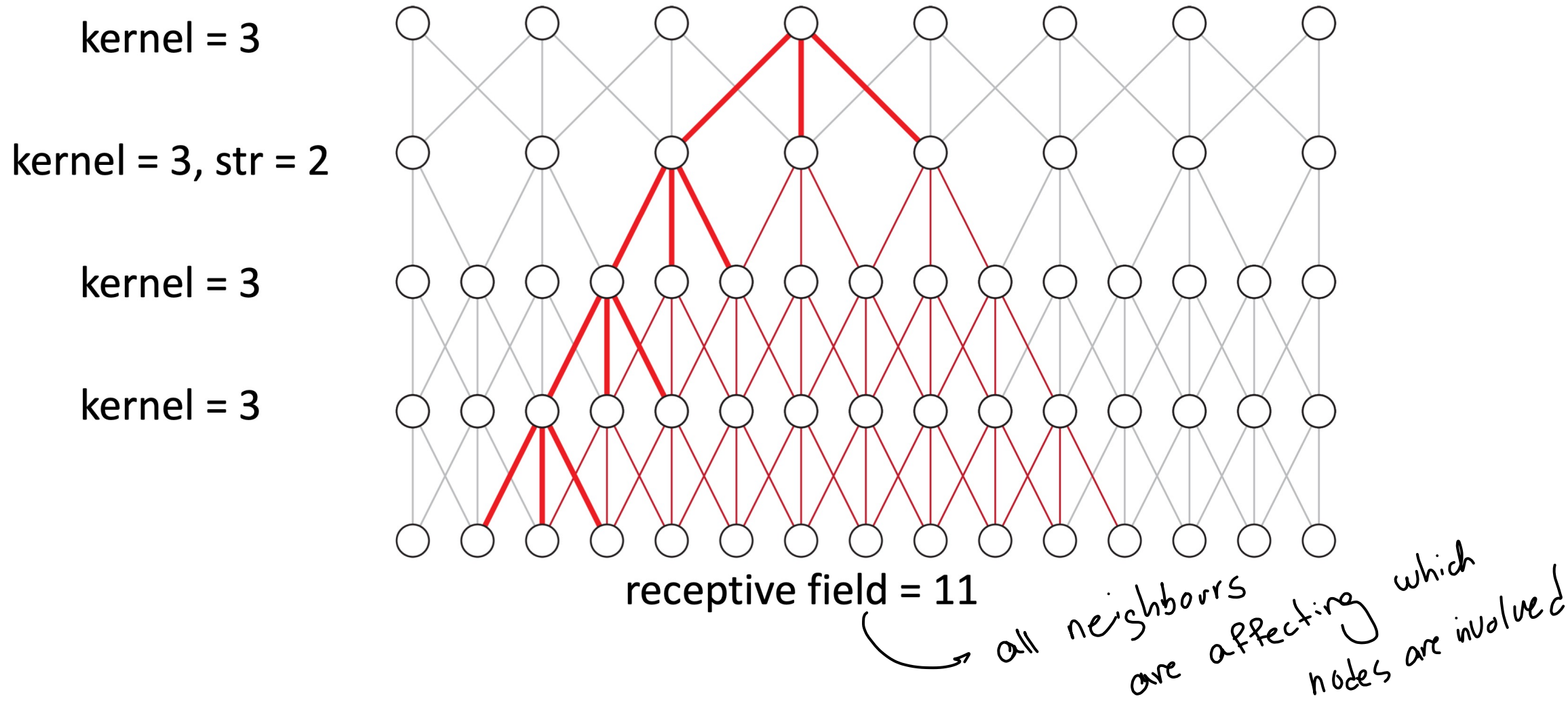
kernel = 3



receptive field = 7

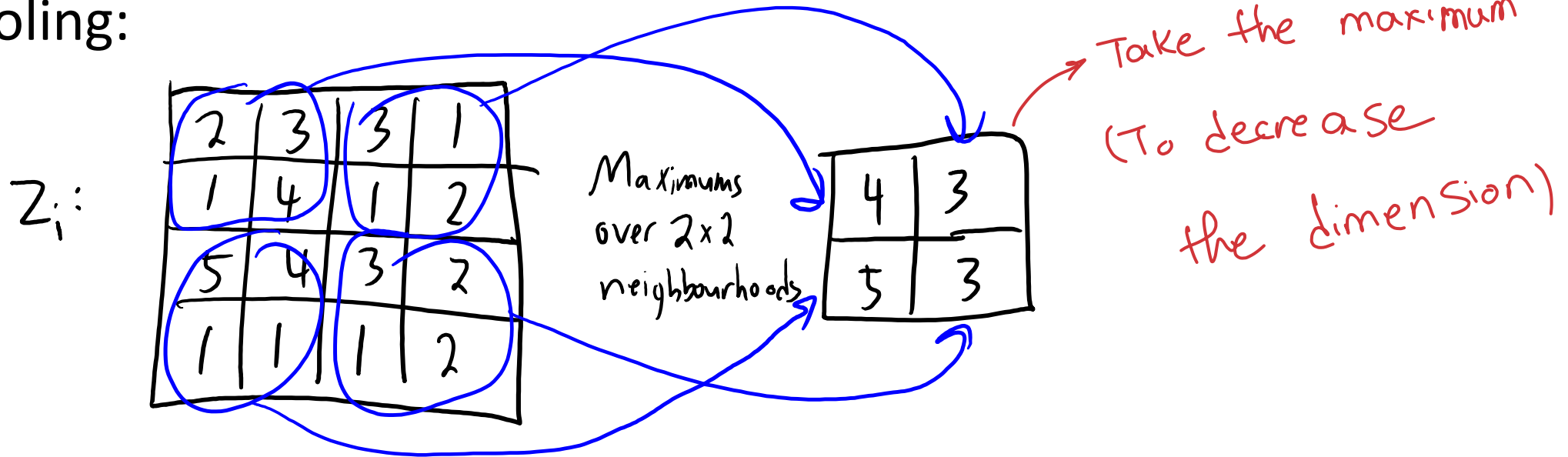
Receptive Fields

Composing many conv layers can create a large receptive field.



Max Pooling Example

- Max pooling:



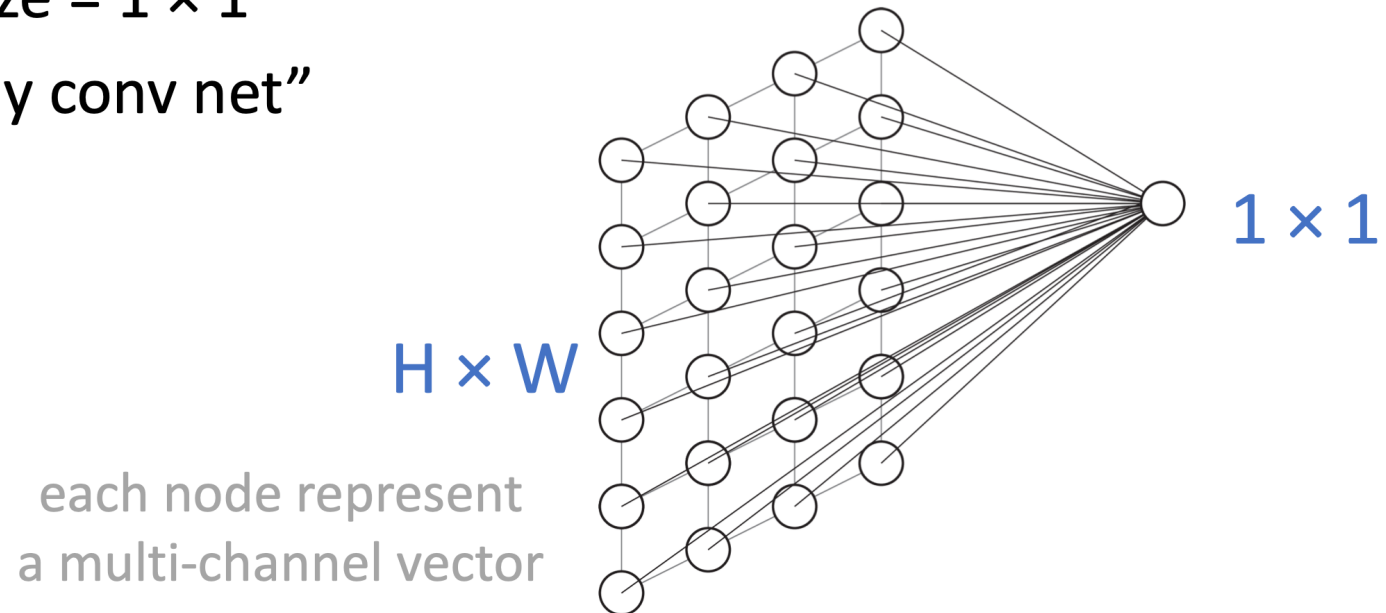
- Decreases size of hidden layer, which speeds up calculations.
- No learnable parameters, introduce spatial invariance
- This is **continuous and piecewise-linear** but **non-differentiable**.
 - Like ReLU, we can still optimize this type of objective with SGD.

Fully Connected Layer in CNN

- every output is connected to every input

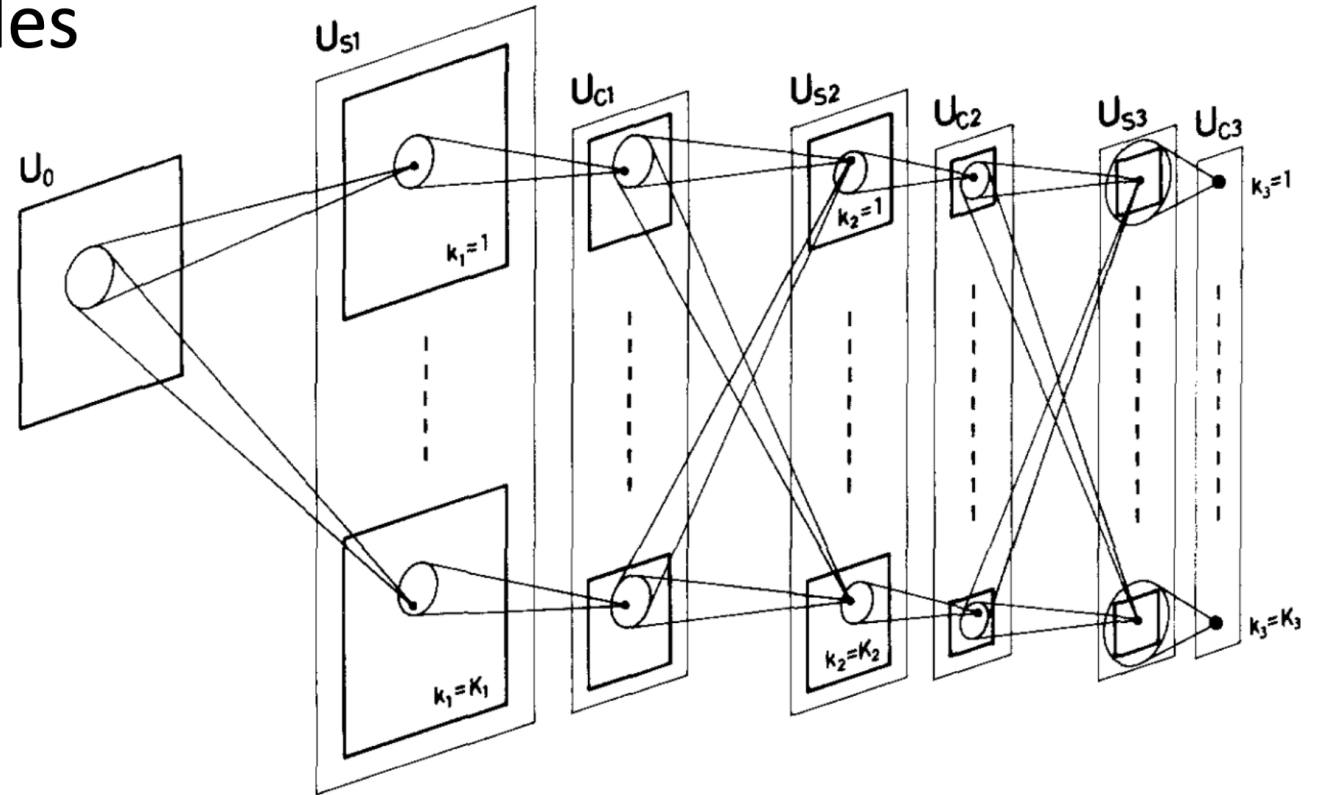
or alternatively:

- convolution with kernel_size = $H \times W$ (w/o padding)
- output map size = 1×1
- useful for “fully conv net”



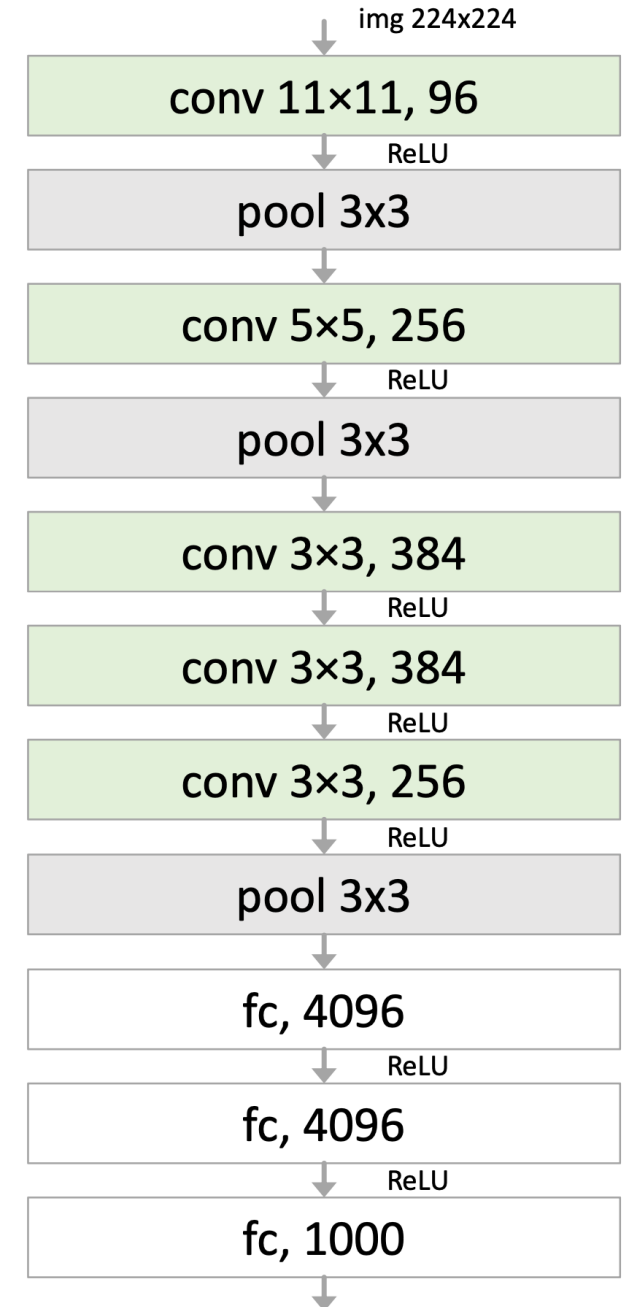
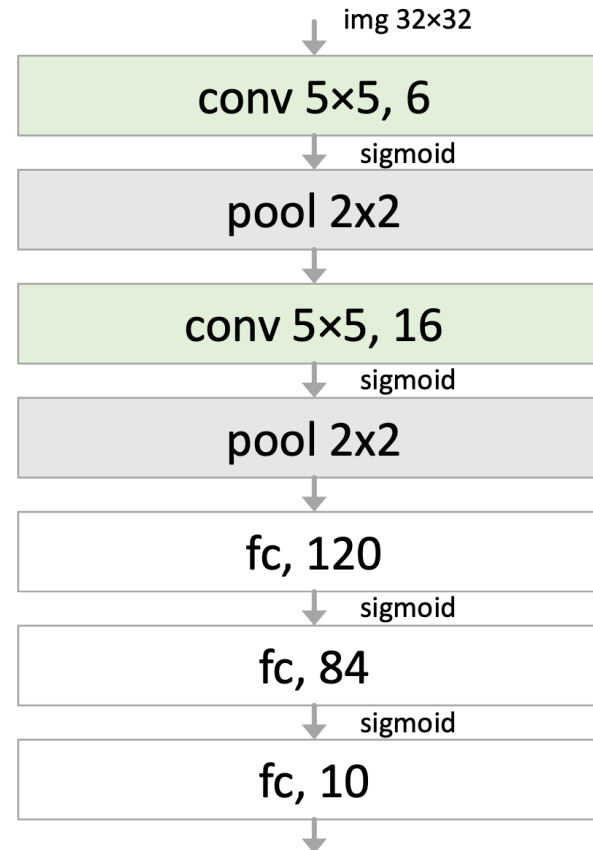
Neocognitron, 1979

- root of ConvNet
- sliding-window features
- composing basic modules
- no backprop



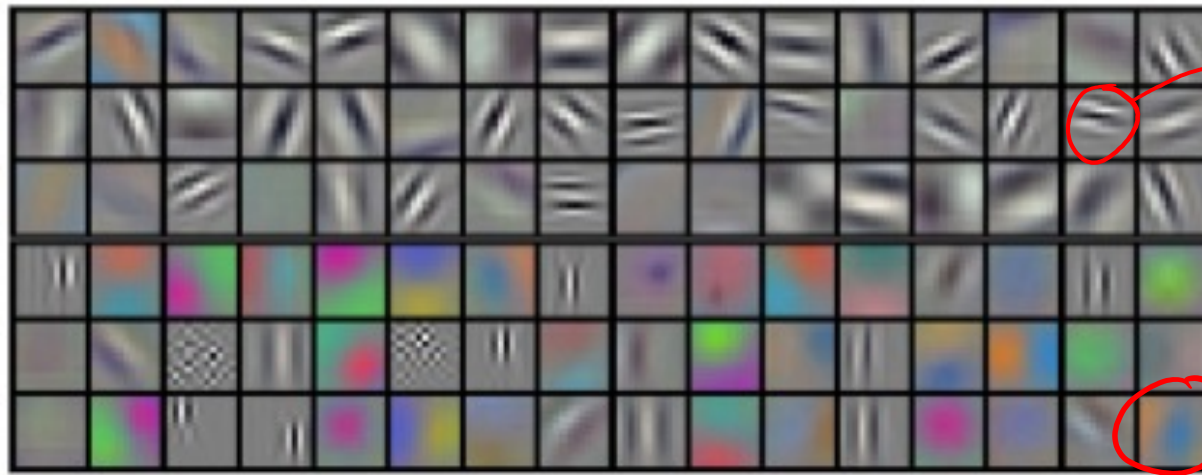
LeNet vs. AlexNet

- deeper (more layers)
- wider (more channels)
- ReLU (not sigmoid)



Are CNNs learning something sensible?

- Filters learned by first layer of original AlexNet (first CNN winner):



"Gabor" filters:

- Gaussian times
sine or cosine.

"Opponent" colour coding.

Figure 3: 96 convolutional kernels of size $11 \times 11 \times 3$ learned by the first convolutional layer on the $224 \times 224 \times 3$ input images. The

- Many single-layer models give similar results.

Discussion of CNNs

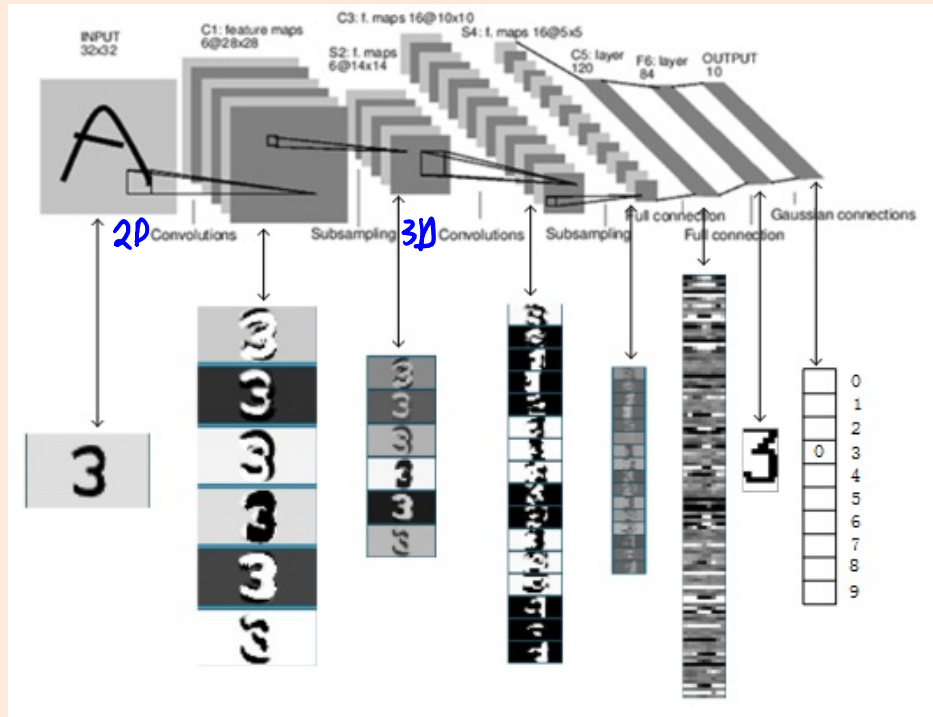
- Convolutional layers reduce the number of parameters in several ways:
 - Each hidden unit only depends on small number of inputs from previous layer.
 - We use the same filters across the image.
 - So we do not learn a different weight for each “connection” like in classic neural networks.
 - Pooling layers decrease the image size.
- CNNs give some amount of translation invariance:
 - Because same filters used across the image, they can detect a pattern anywhere in the image.
 - Even in image locations where the pattern has never been seen.
- CNNs are not only for images!
 - Can use CNNs for 1D sequences like time-series, sound, language, or biological sequences.
 - Can use CNNs for 3D objects like videos or medical image volumes.
 - Can use CNNs for graphs.
- But you do need some notion of “neighbourhood” for convolutions to make sense.

Summary

- Convolutional neural networks:
 - Include layers that apply several (learned) convolutions.
 - Significantly decreases number of parameters.
 - Achieves a degree of translation invariance.
 - Often combined with pooling operations like max pooling.
- CNNs seem to be learning sensible things.
 - Earlier layers seem to represent low-level features.
 - Later layers seem to represent complex object-level features.

LeNet Convolutional Neural Networks

- Classic **convolutional neural network** (LeNet):



- Visualizing the “activations”:

- [An Interactive Node-Link Visualization of Convolutional Neural Networks \(adamharley.com\)](http://adamharley.com)
- <http://cs231n.stanford.edu>

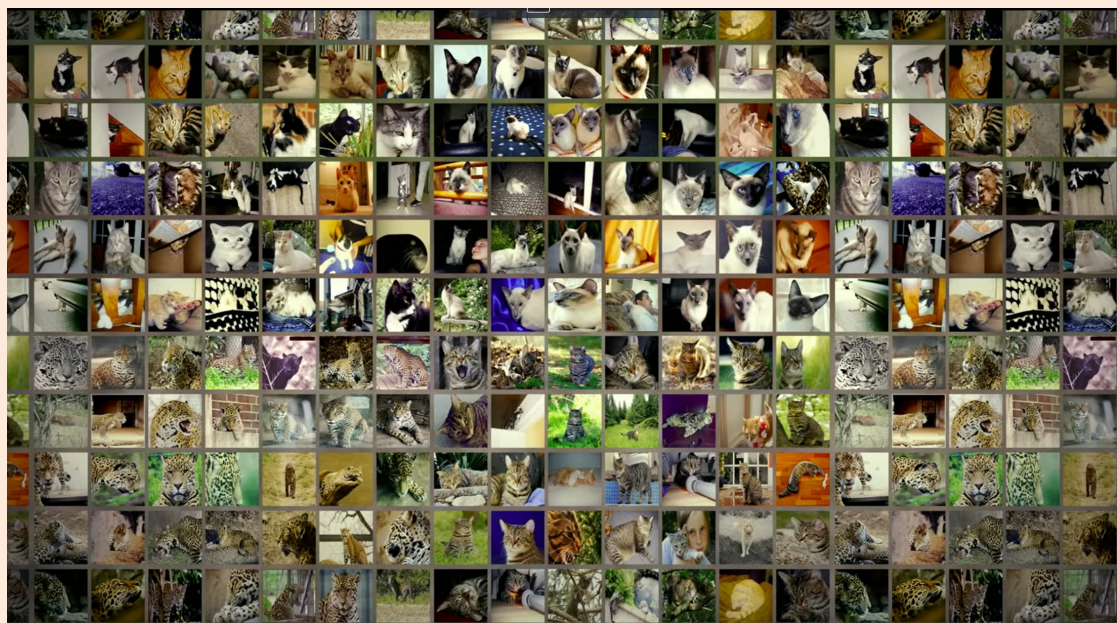
softmax
2 'fully-connected'
max pooling
3D convolutions
max pooling

2D convolutions

(from 3-4 previous layer images)

ImageNet Competition

- **ImageNet**: Millions of labeled images, 1000 object classes.
 - Task is to classify images into one of the 1000 class labels.
 - Everyone submits their “best” model, winners announced.
 - Humans error level estimated at ~5%.



Syberian Husky



Canadian Husky

AlexNet Convolutional Neural Network

- Modern CNN era started with **AlexNet** (won 2012 competition):
 - 15.4% error vs. 26.2% for closest competitor.
 - 5 convolutional layers.
 - 3 fully-connected layers.
 - SG with momentum.
 - ReLU non-linear functions.
 - Data translation/reflection/cropping.
 - L2-regularization + Dropout.
 - 5-6 days on two GPUs.

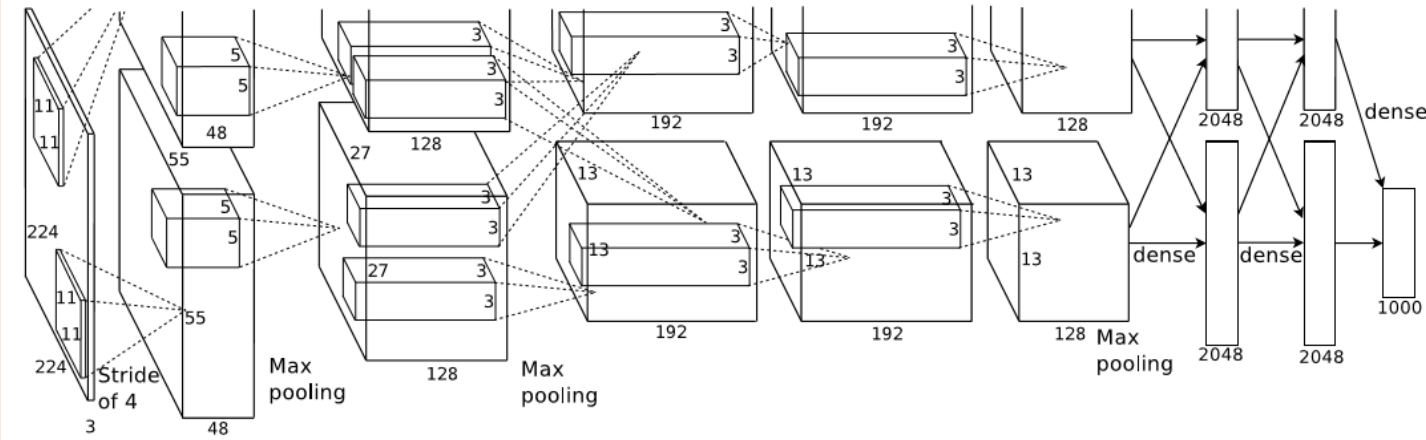
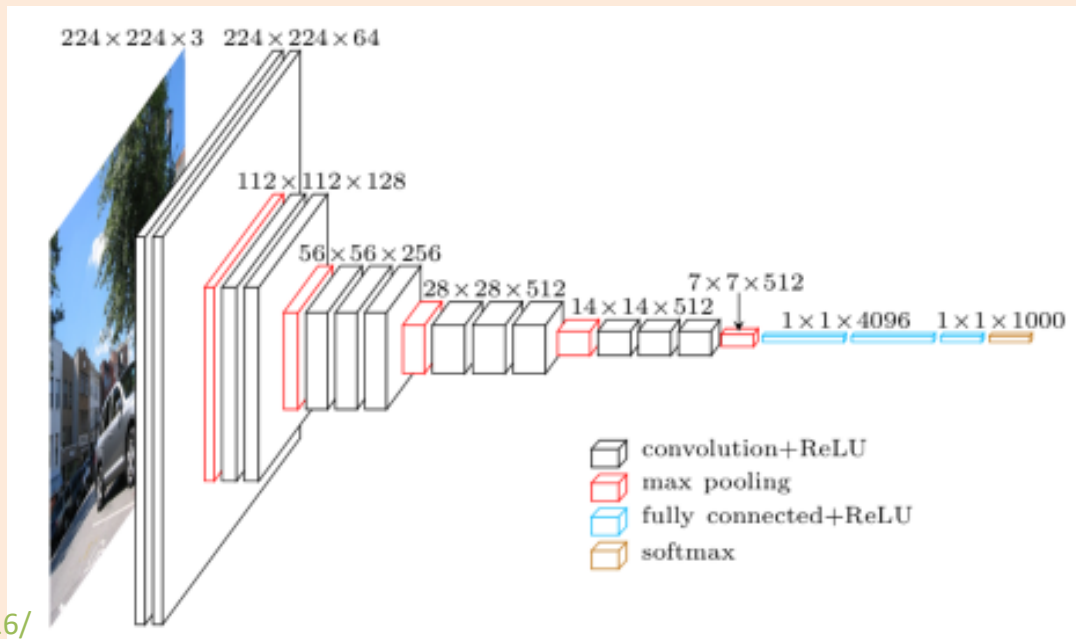


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

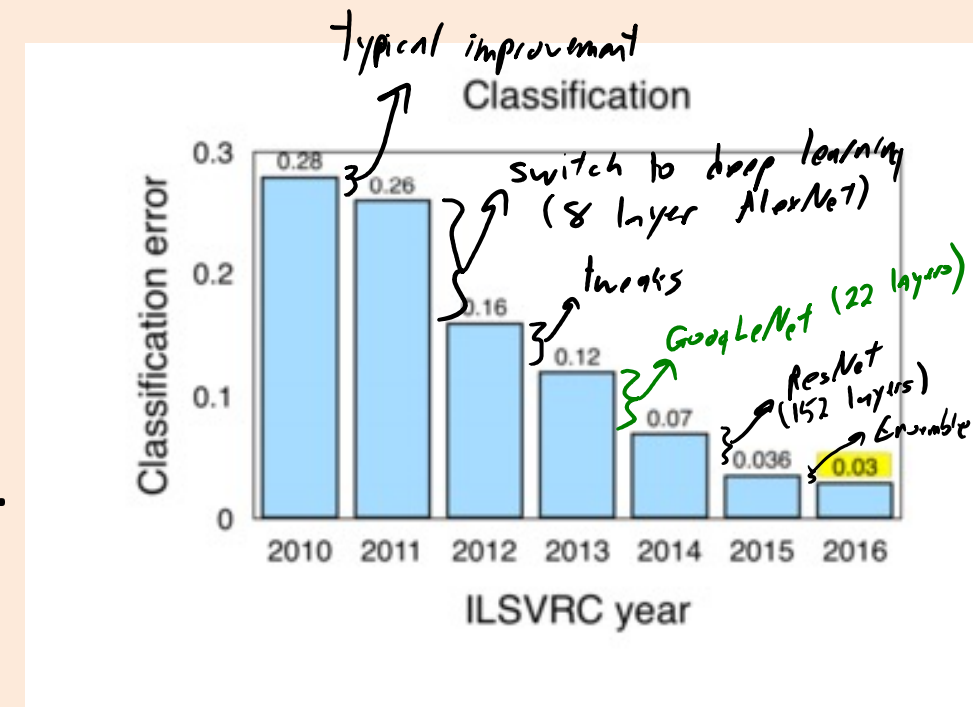
ImageNet Insights

- Filters and stride got smaller over the years.
 - Popular VGG approach uses **3x3 convolution layers** with **stride of 1**.
 - 3x3 followed by 3x3 simulates a 5x5, and another 3x3 simulates a 7x7, and so on.
 - Speeds things up and reduces number of parameters.
 - Also increases number of non-linear ReLU operations.



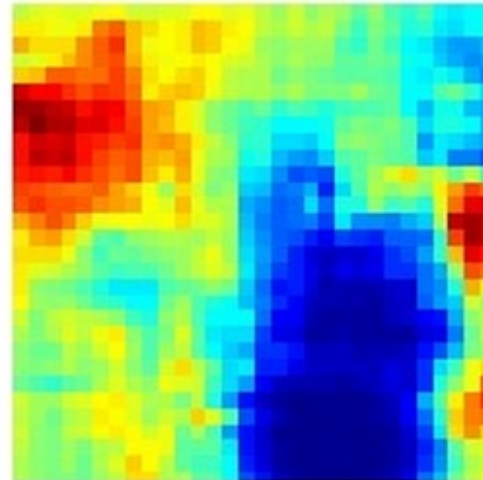
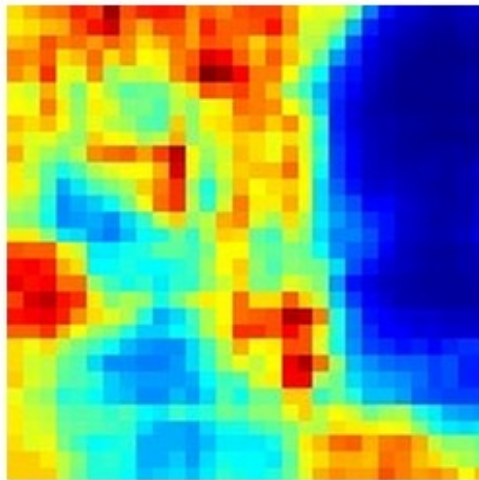
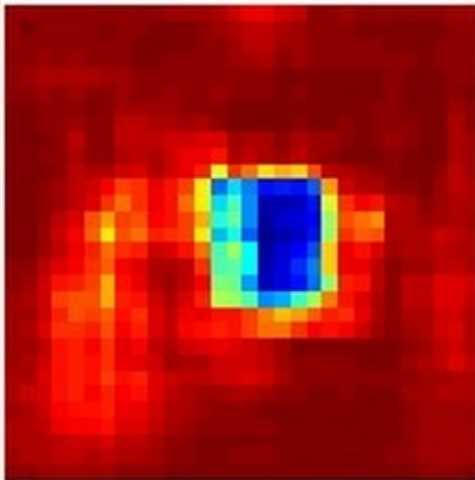
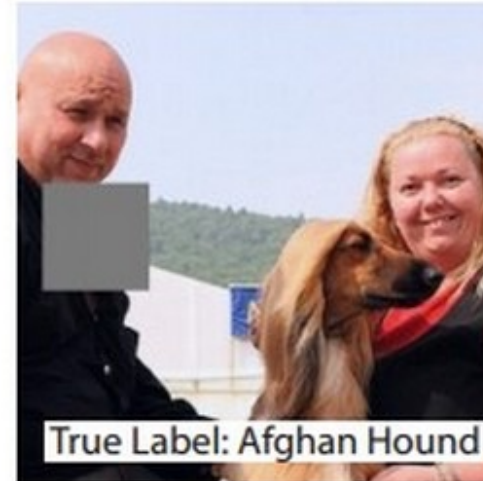
ImageNet Insights

- Filters and stride got smaller over time.
 - Popular VGG approach uses 3x3 convolution layers with stride of 1.
 - GoogLeNet used multiple filter sizes (“inception layer”), but not as popular.
- Eventual switch to “fully-convolutional” networks.
 - No fully connected layers.
- ResNets introduced in 2015.
 - Won all 5 tasks with 152 layers.
 - Trained for 2-3 weeks on 8 GPUs.
- Ensembles help.
 - 2016 winner: ensemble of previous networks.
- Competition stopped in 2017!



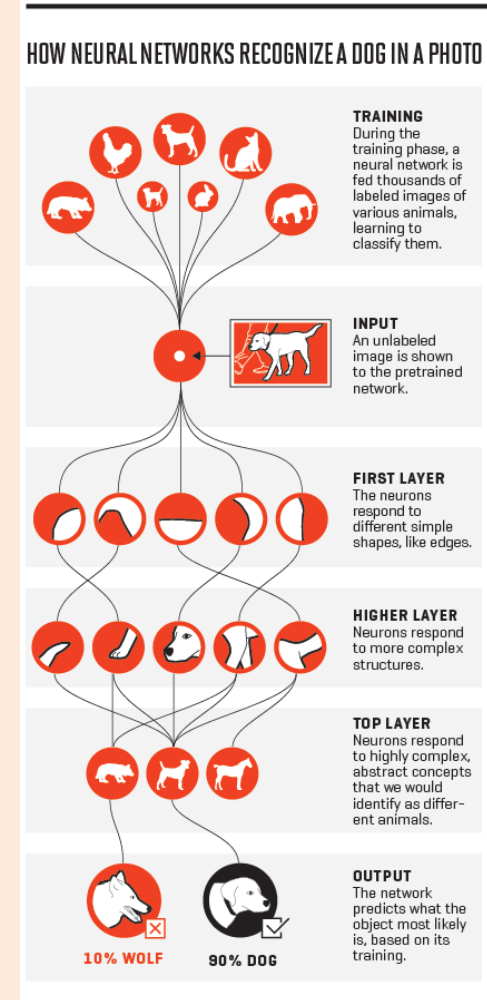
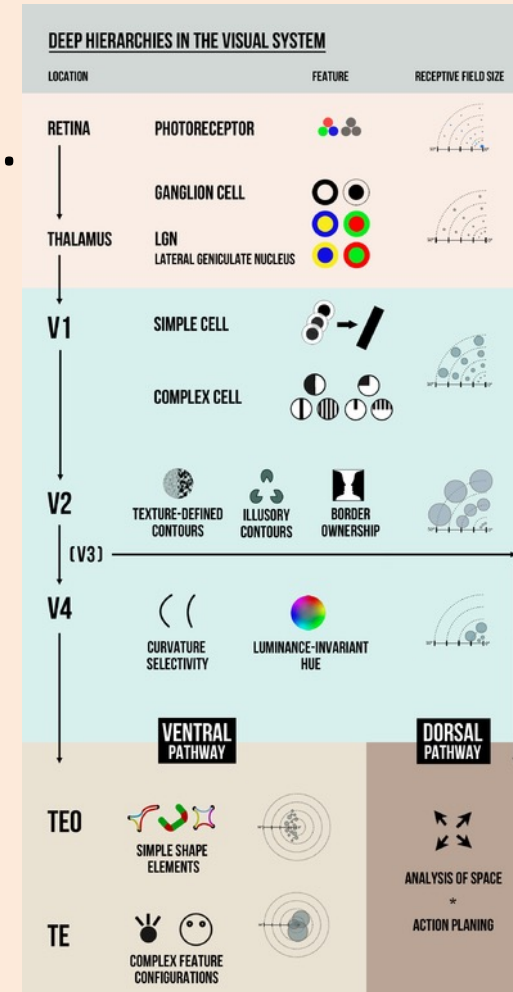
Are CNNs learning something sensible?

- We can look at how prediction changes if we hide part of image:



Are CNNs learning something sensible?

- Recall that deep learning and CNNs are **motivated by ideas about human vision**.
 - First layers detect simple features like Gaussians and Gabors filters.
 - Later layers detect more complicated features like corners, repeating patterns.
 - Deeper layers starts to recognize complex parts of objects.
 - Deepest layers recognize full object concepts.
- Is this what trained CNNs actually do?

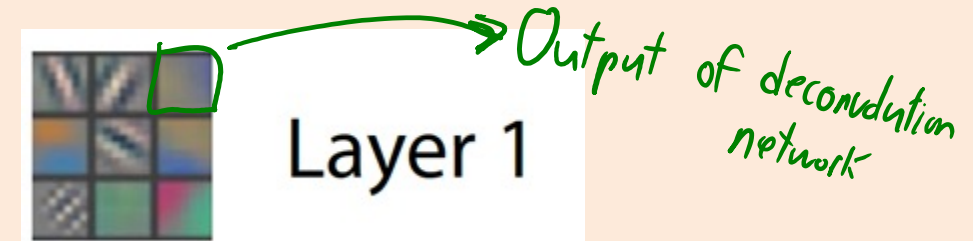


Are CNNs learning something sensible?

- It is **harder to visualize what is learned in other layers.**

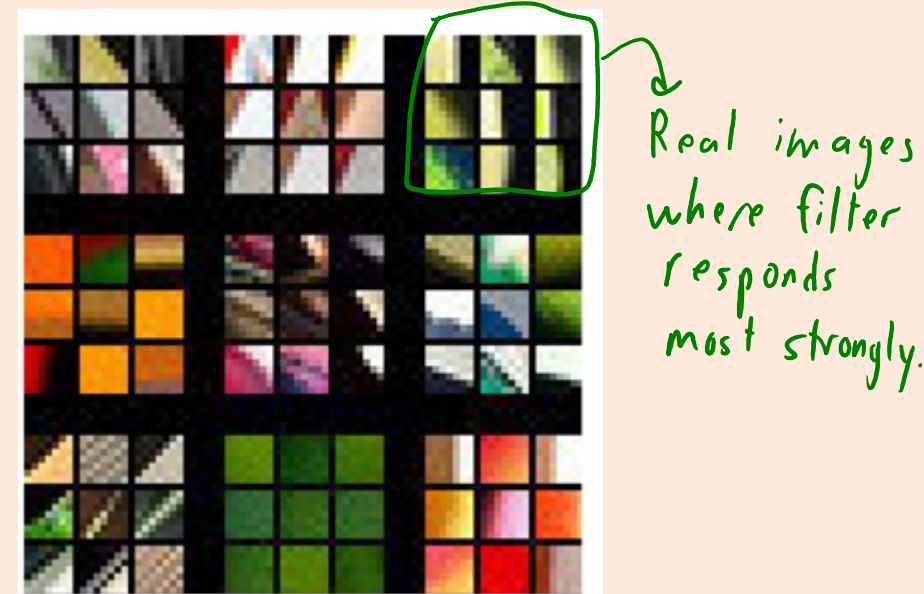
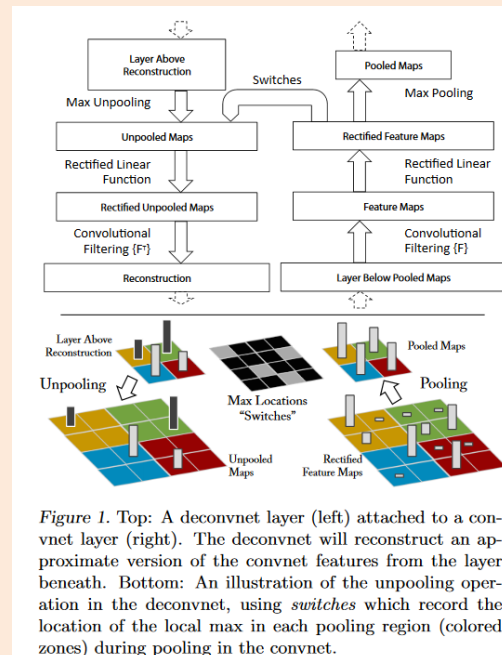
- Approach 1:

- Search for training data **image patches that maximally-activates** a filter.
- Then try to reason about what the filter is doing.

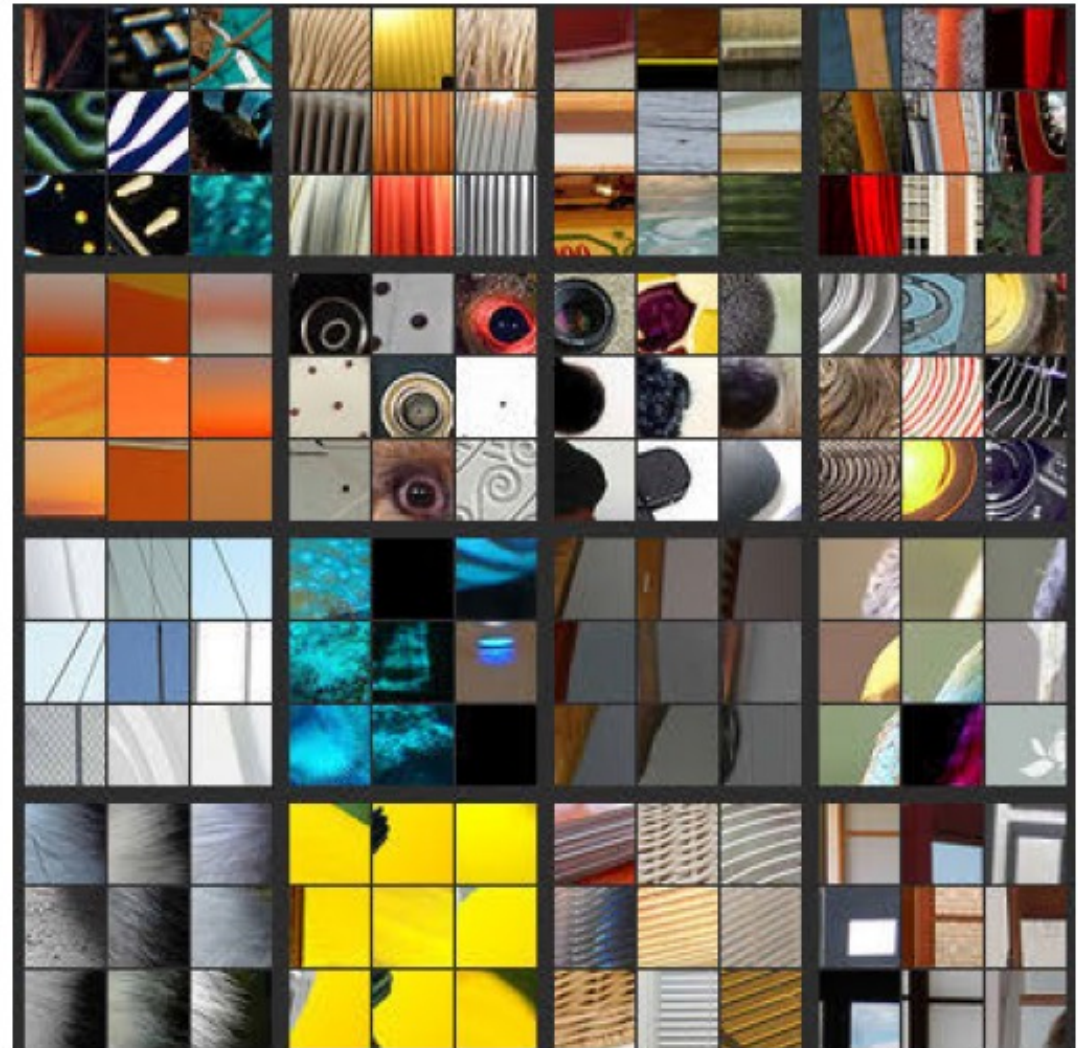
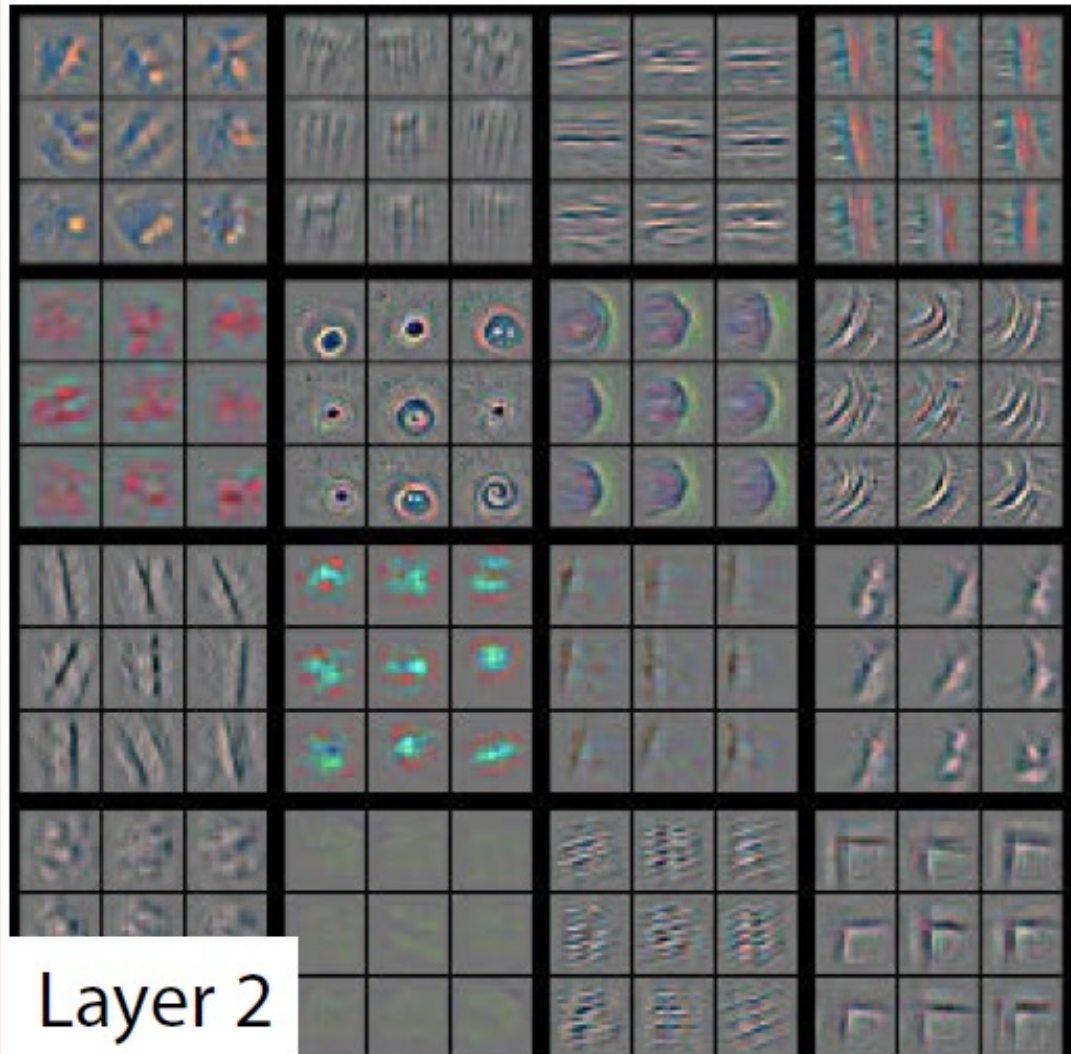


- Approach 2:

- Apply **deconvolution network to these patches** to try to “reverse” the operations.
- Uses “transposed convolutions” and “unpooling” to **visualize “what activated the filter”**.



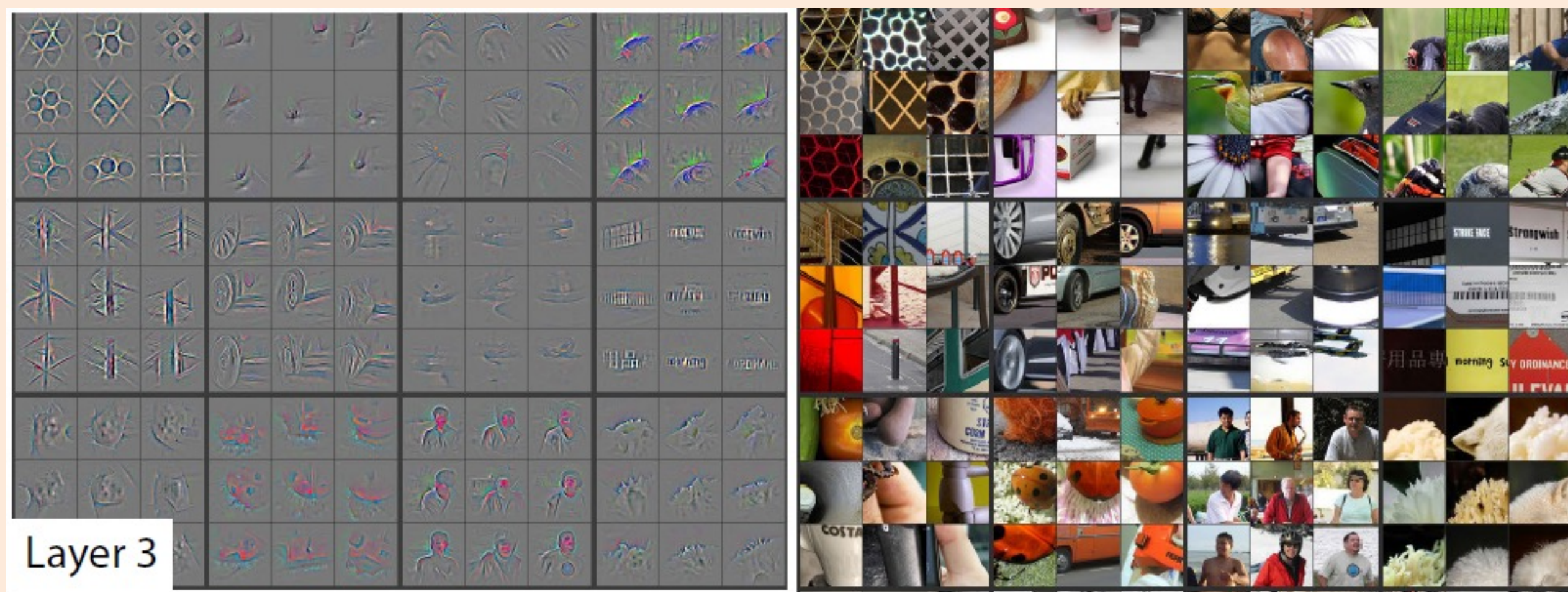
Are CNNs learning something sensible?



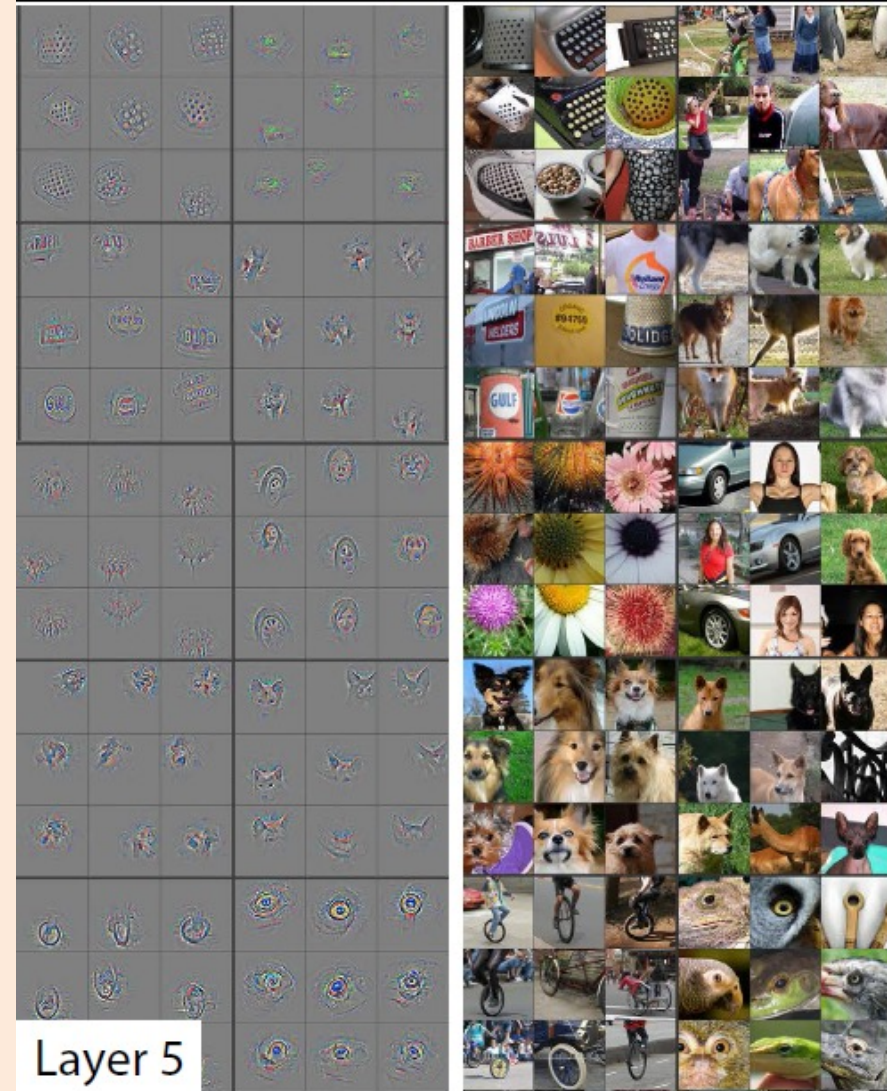
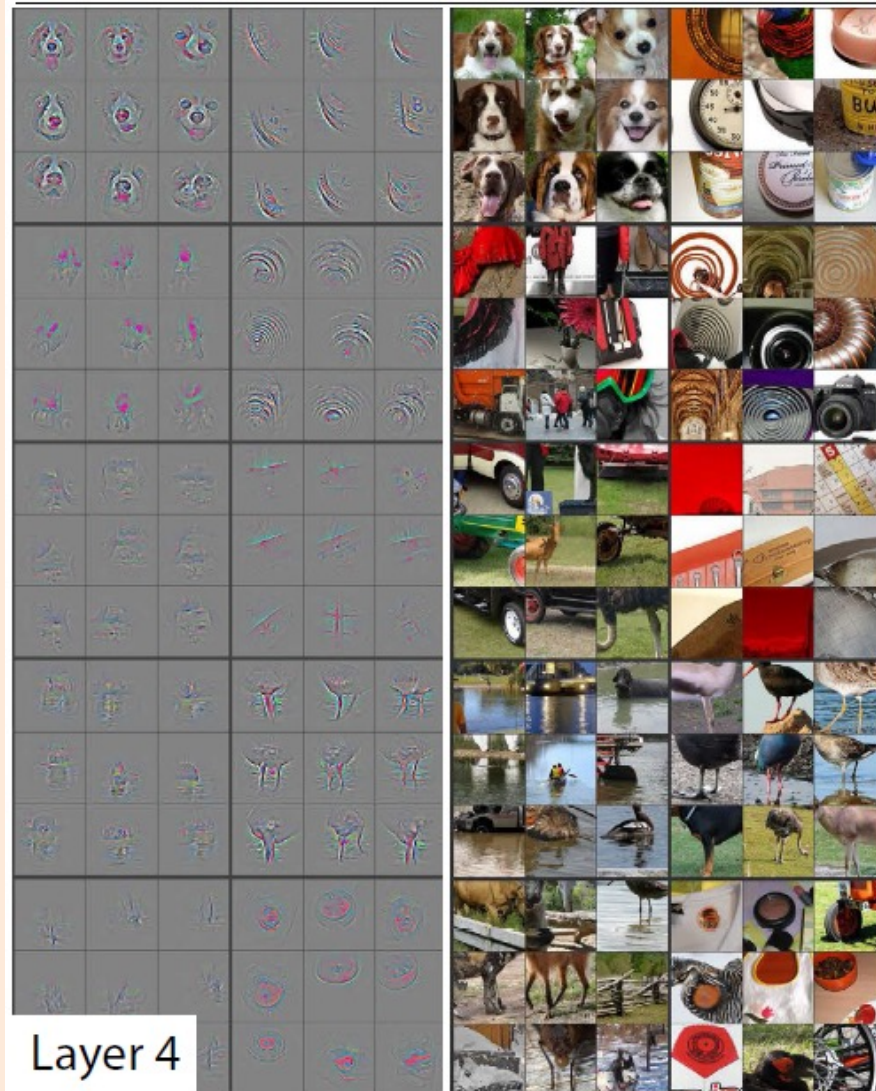
Patch
from
data
giving
largest
response

Result of deconvolution network

Are CNNs learning something sensible?



Are CNNs learning something sensible?



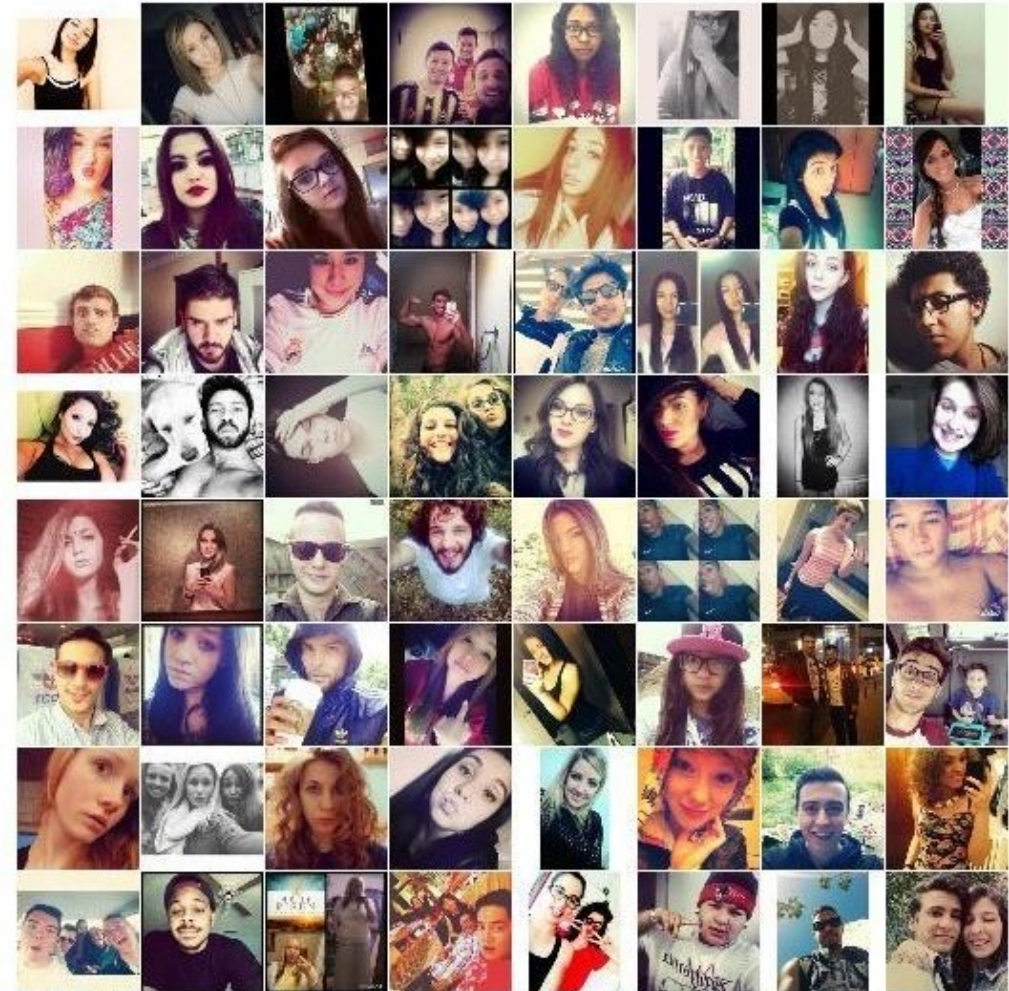
CNNs for Rating Selfies

Our training data

Bad selfies



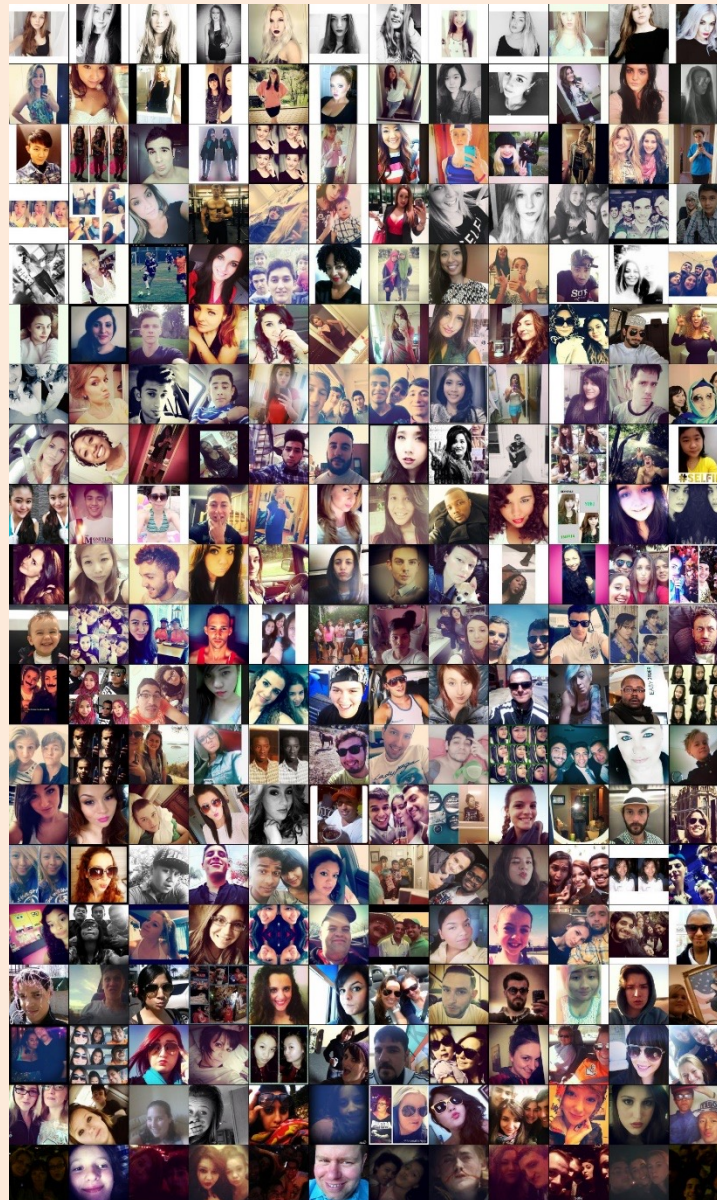
Good selfies



CNNs for Rating Selfies

Do: ↙

- Be female
- Have face be $\frac{1}{3}$ of image
- Cut off forehead
- Show long hair
- Oversaturate face
- Use filter
- Add border



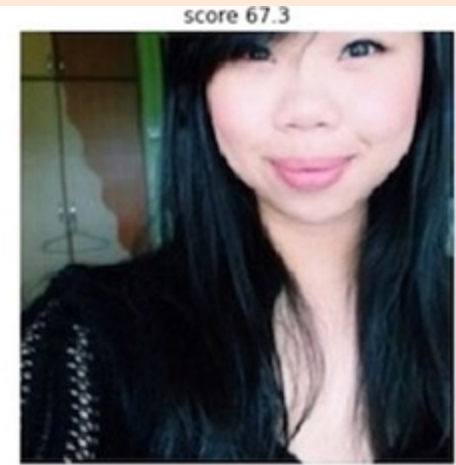
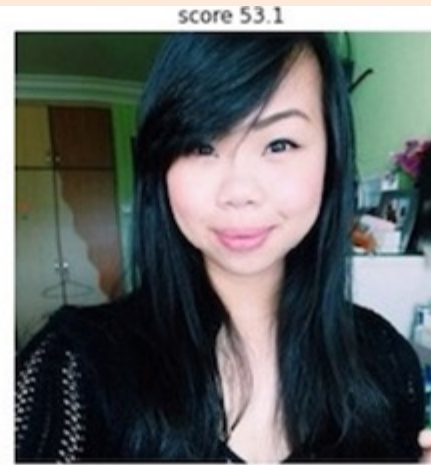
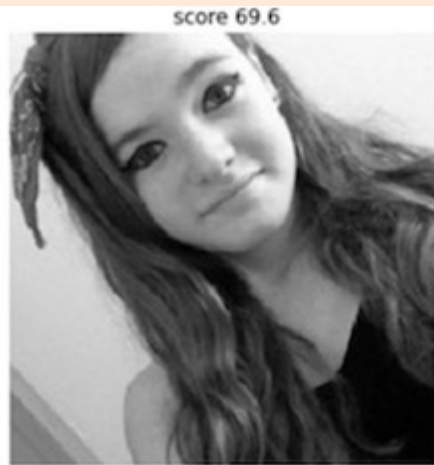
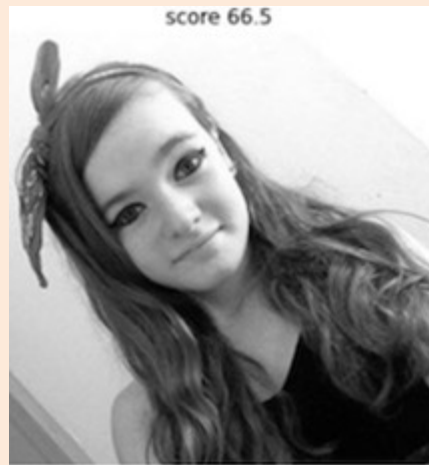
Don't:

- Use low lighting
- Make head too big
- Take group shots



CNNs for Rating Selfies

Finding best
image crop:



CNNs for Choosing YouTube Thumbnails

